


**Sri Lanka Institute of Advanced Technological Education (SLIATE)**

**Advanced Technological Institute - Kurunegala**

**Higher National Diploma in Information Technology**

**Batch-2324(FT)**

# Project Report

# AgroSmart Marketplace: A Smart Agriculture Trading Platform

## IT4052 | Programming Individual Project

**Supervisor**

**Ms.K.G.D.De.A Wijesinghe**

**N.M.T.H Senevirathna**

**KUR/IT/2324/F/0013**

**Academic Year : 2324**

**Year: 2nd Year II Semester**

# Declaration

I hereby certify that this project report, entitled **"AgroSmart Marketplace,"** is entirely my own work, carried out under the supervision of **Ms.K.G.D.De.A Wijesinghe**, and has not been submitted, either in whole or in part, for any other academic award.

Any information or material obtained from other sources has been properly acknowledged and referenced. This document is the property of the Sri Lanka Institute of Advanced Technological Education (SLIATE), and no part of it may be reproduced or transmitted by any means without the prior written permission of the author.

**Signature of Candidate:** .....

**Name:** N.M.T.H. Senevirathna

**Registration Number:** KUR/IT/2324/F/0013

**Date:** .....…

**Certified by**

**Supervisor's Name:** Ms.K.G.D.De.A Wijesinghe

**Signature:** .....

**Date:** .....

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of the AgroSmart Marketplace project.

First, I extend my deepest appreciation to my project supervisor, **Ms. K.G.D.De.A. Wijesinghe**, for her valuable guidance, constructive feedback, encouragement, and continuous support during every stage of this project. Her advice helped me to improve the project concept, identify weaknesses, and develop the system in a structured and effective manner.

I am also grateful to the academic staff of the **Higher National Diploma in Information Technology programme at the Advanced Technological Institute, Kurunegala**, for providing the knowledge and technical foundation required to complete this project. The lessons and practical experience gained throughout the programme contributed significantly to the analysis, design, development, testing, and documentation of the system.

I would also like to thank the farmers, buyers, and other individuals who provided useful ideas and information regarding the challenges associated with agricultural trading. Their observations helped me to understand the practical difficulties faced by farmers and buyers and supported the identification of important system requirements.

Furthermore, I sincerely appreciate my classmates and friends for their suggestions, encouragement, and assistance during the development and testing of the system. Their feedback helped me to identify usability issues and improve the overall quality of the AgroSmart Marketplace.

Finally, I am deeply thankful to my family for their patience, understanding, motivation, and continuous support throughout the project. Their encouragement gave me the confidence and determination required to complete this project successfully.

# ABSTRACT

Agricultural trading in Sri Lanka often depends heavily on intermediaries, which can reduce price transparency, limit direct communication, and prevent farmers from receiving fair returns for their products. Buyers may also experience difficulties in locating suitable agricultural products, obtaining reliable product information, and negotiating prices directly with farmers. In addition, the lack of demand-related information can make it difficult for farmers to make effective production, pricing, and selling decisions. The AgroSmart Marketplace was developed to address these challenges by providing a web-based platform that supports direct, transparent, and data-driven interaction between farmers and buyers.

The system provides separate role-based functionalities for buyers, farmers, and administrators. Farmers can publish and manage agricultural products, define selling and minimum prices, receive smart price recommendations, review buyer offers, submit counter-offers, manage orders, and view product demand information. Buyers can browse and search products, filter products according to district and city, discover nearby products, view detailed product information, submit price offers, respond to counter-offers, track orders, and provide ratings and feedback after successful transactions. The negotiation mechanism allows farmers to accept, reject, or counter buyer offers, while minimum-price validation protects farmers from offers below an acceptable value.

The system also collects buyer activities, including product searches, views, offers, and completed orders. A rule-based demand analysis mechanism uses these activities to calculate demand scores and classify products as High Demand or Low Demand. The smart pricing mechanism considers product demand, completed-order information, marketplace prices, and the farmer's minimum price to recommend a suitable price range. Administrators can manage user accounts, monitor product listings, remove or restore inappropriate products, review administrative activity logs, and generate sales and order reports in PDF format.

AgroSmart Marketplace was developed using Laravel, PHP, MySQL, Tailwind CSS, Alpine.js, Laravel Jetstream, and Fortify. The completed system demonstrates that a structured digital marketplace can improve direct communication, price negotiation, location-based product discovery, transparency, and informed decision-making within agricultural trading.

**Keywords:** Smart agriculture, digital marketplace, price negotiation, demand analysis, smart pricing, location-based matching.

## Table of Contents

ACKNOWLEDGEMENT....3
ABSTRACT....4
CHAPTER 1 – INTRODUCTION....8
1.1. Purpose....8
1.2. System Overview....9
1.3. Problem Statement....10
1.4. Goal and Vision....11
CHAPTER 2 – REQUIREMENT SPECIFICATION....12
2.1. Requirement Gathering Process....12
2.2. System Functions....13
2.2.1. Common Functions....13
2.2.2. Farmer Functions....13
2.2.3. Buyer Functions....13
2.2.4. Administrator Functions....14
2.2.5. Smart System Functions....14
2.3. Software Requirements....14
2.4. Hardware Requirements....15
2.4.1. Development Hardware....15
2.4.2. User Hardware....15
CHAPTER 3 – SYSTEM DESIGN....16
3.1. Frontend Design....17
3.2. Database Design....37
3.2.1. Database Tables....37
3.2.2. Database Relationship....41
3.2.3. Entity Relationship Diagram....42
3.3. Architectural Design....43
3.3.1. Model–View–Controller (MVC)....43
3.3.2. Three-Tier Architecture....44
3.4. Use Case Diagram....47
3.5. UML Class Diagram....48
CHAPTER 4 – SYSTEM TESTING....49
4.1. Introduction....49

4.2. Functional Testing....49
4.3. Validation Testing....52
4.4. Security Testing....52
4.5. Usability and Responsive Testing....53
CHAPTER 5 – CONCLUSION AND FUTURE WORK....54
5.1. Conclusion....54
5.2. Future Work....54
References....55

# Table of Figures

Figure 3.1.1: Public Landing Page – Hero Section.....18
Figure 3.1.2: Login Interface.....19
Figure 3.1.3: User Registration Interface.....20
Figure 3.1.4: Buyer Dashboard Interface.....21
\Figure 3.1.5: Product Browsing.....22
Figure 3.1.6: Product Details.....23
Figure 3.1.7: Offer Submission Interface.....23
Figure 3.1.8: Buyer Offer Management Interface.....24
Figure 3.1.9: Buyer Order Tracking and Review Interface.....25
Figure 3.1.10: Farmer Dashboard Interface.....26
Figure 3.1.11: Product Publishing and Smart Pricing Interface.....27
Figure 3.1.12: Farmer Product Management Interface.....28
Figure 3.1.13: Farmer Offer Management Interface.....29
Figure 3.1.14: Farmer Order Management Interface.....30
Figure 3.1.15: Demand Analysis Interface.....31
Figure 3.1.16: Administrator Dashboard Interface.....32
Figure 3.1.17: User Management Interface.....33
Figure 3.1.18: Product Management Interface.....34
Figure 3.1.19: Administrative Activity Logs Interface.....35
Figure 3.1.20: Sales and Orders Report Interface.....36
Figure 3.2.1: Entity Relationship Diagram.....42
Figure 3.3.1: MVC and Three-Tier Architecture.....46
Figure 3.4.1: Use Case Diagram.....47
Figure 3.5.1: Class Diagram.....48

# Index of Tables

Table 2.3.1: Software Requirements.....15
Table 3.2.1: Database Relationships.....41
Table 4.2.1: User Registration and Login Test Cases.....49
Table 4.2.2: Product Management Test Cases.....50
Table 4.2.3: Offer Negotiation Test Cases.....50
Table 4.2.4: Order and Review Test Cases.....51
Table 4.2.5: Administrator Test Cases.....51
Table 4.3.1: Validation Test Cases.....52
Table 4.4.1: Security Test Cases.....52
Table 4.5.1: Usability and Responsive Test Cases.....53

# CHAPTER 1 – INTRODUCTION

Agriculture plays an important role in Sri Lanka's economy, but traditional trading methods create challenges for farmers and buyers, including limited market information, unfair pricing, and difficulties in finding suitable products and trading partners.

AgroSmart Marketplace addresses these challenges by directly connecting farmers and buyers through a web-based platform. It provides smart pricing, demand analysis, location-based matching, offer negotiation, order management, and administrative monitoring to improve transparency and decision-making.

## 1.1. Purpose

The primary purpose of the AgroSmart Marketplace is to provide a centralized and user-friendly digital platform where farmers can publish agricultural products and buyers can discover and purchase those products through a structured offer-based process.

Farmers can publish product details such as the product name, category, available quantity, selling price, minimum acceptable price, location, image, and description. Buyers can browse available products, search by product or farmer information, filter products according to district and city, and view detailed product information before submitting an offer.

The system allows farmers and buyers to negotiate prices without depending entirely on intermediaries. A buyer can submit an offer for a required quantity, while the farmer can accept, reject, or respond with a counteroffer. When an offer is accepted, the system creates an order that can be tracked through several stages until completion.

The platform also supports farmers by providing smart price recommendations based on available marketplace data, completed-order price history, product demand, and the farmer's minimum price. Demand analysis helps farmers understand whether their products currently have high or low demand. These functions enable farmers to make more informed product and pricing decisions.

## 1.2. System Overview

AgroSmart Marketplace is a role-based web application consisting of three main user roles: Buyer, Farmer, and Administrator. Each role is provided with a separate dashboard and functions relevant to its responsibilities.

A farmer can publish and manage agricultural products, define minimum prices, receive smart price recommendations, review buyer offers, submit counteroffers, confirm orders, update order statuses, and view demand analysis information. The smart pricing mechanism provides a suggested price range using relevant completed orders and marketplace product prices while ensuring that the recommendation does not fall below the farmer's minimum price.

A buyer can browse agricultural products, search for specific products, filter products by district or city, and receive nearby product recommendations according to the buyer's registered location. After viewing product details, the buyer can submit an offer to the farmer. Buyers can monitor offer status, accept or reject counteroffers, track confirmed orders, and provide a rating and written review after an order has been completed.

The administrator monitors the overall platform through an administrative dashboard. Administrators can view and manage registered user accounts, block or reactivate buyers and farmers, monitor product listings, remove inappropriate listings, restore removed products, and review important administrative activity logs. The system also provides a sales and orders report that can be filtered and downloaded as a PDF document.

The application was developed using Laravel, PHP, MySQL, Tailwind CSS, Alpine.js, Laravel Jetstream and Fortify. It follows the Model–View–Controller architectural pattern to separate the data, interface, and application logic. The completed system is deployed through Railway, allowing it to be accessed online.

## 1.3. Problem Statement

Many agricultural transactions in Sri Lanka continue to depend on traditional communication methods and intermediaries. As a result, farmers may not have direct access to a wider group of buyers and may receive prices that do not accurately reflect current demand or market conditions. A lack of reliable pricing information can make it difficult for farmers to determine an appropriate selling price while protecting their minimum expected income.

Buyers also experience challenges when searching for agricultural products. Product availability, farmer location, quantity, and pricing details may be distributed across different communication channels or may not be available at the required time. This makes product comparison and identification of nearby farmers difficult and time-consuming.

Traditional trading processes may also lack a structured mechanism for submitting offers, negotiating prices, tracking orders, and recording completed transactions. Without a centralized system, both farmers and buyers may find it difficult to monitor the progress of negotiations and orders. Furthermore, platform administrators require a reliable method to monitor user accounts, product listings, administrative activities, sales, and orders.

Therefore, a digital solution is required to connect farmers and buyers directly, organize product and transaction information, support transparent negotiation, provide data-based pricing and demand insights, recommend products according to location, and allow administrators to maintain the reliability of the platform.

## 1.4. Goal and Vision

The main goal of the AgroSmart Marketplace is to develop a secure, accessible, and intelligent agricultural trading platform that improves direct interaction between farmers and buyers. The system aims to simplify product publishing, product discovery, price negotiation, order management, and marketplace monitoring.

The specific goals of the system are:

- To enable farmers to publish and manage agricultural products efficiently.
- To allow farmers to specify a minimum acceptable price for each product.
- To provide smart price recommendations using relevant marketplace and completed-order information.
- To identify product demand using buyer searches, product views, submitted offers, and completed orders.
- To allow buyers to discover products according to their district and city.
- To provide a structured offer, counteroffer, acceptance, and rejection process.
- To support order tracking from confirmation to successful completion.
- To enable buyers to provide ratings and feedback for completed orders.
- To allow administrators to manage users, products, activities, and reports.

The vision of AgroSmart Marketplace is to contribute to a more connected, transparent, and data-informed agricultural marketplace in Sri Lanka. By reducing communication barriers and improving access to useful market information, the platform seeks to create fairer opportunities for farmers and a more convenient purchasing process for buyers.

# CHAPTER 2 – REQUIREMENT SPECIFICATION

Requirement specification defines the functional and technical requirements needed to develop the AgroSmart Marketplace. These requirements were identified by examining the challenges faced by farmers and buyers, reviewing the proposed system, and analysing the complete agricultural trading workflow.

## 2.1. Requirement Gathering Process

The system requirements were gathered by studying traditional agricultural trading activities and identifying the difficulties faced by farmers and buyers. The requirement-gathering process focused on product publishing, product discovery, price negotiation, order processing, demand analysis, and marketplace administration.

The following methods were used:

- Observation of common agricultural product trading processes.
- Discussions with potential farmers and buyers.
- Review of existing online marketplace systems.
- Analysis of project proposal and SRS documentation.
- Identification of role-specific requirements and workflows.
- Continuous testing and evaluation during development.

The collected requirements were organized according to three primary roles: Farmer, Buyer, and Administrator. The requirements were refined throughout the development process based on system testing and user-interface evaluation.

## 2.2. System Functions

The AgroSmart Marketplace provides separate functions for each user role.

### 2.2.1. Common Functions

- Register as a farmer or buyer.
- Log in using an authorized account and selected role.
- Access a dashboard according to the user role.
- Store the user’s district, city, and contact information.
- Prevent blocked accounts from accessing the platform.
- Log out securely from the system.

### 2.2.2. Farmer Functions

- Publish products with name, category, quantity, price, minimum price, location, image, and description.
- View and manage published products.
- Receive a smart price recommendation based on marketplace and completed-order data.
- View the current demand level of products.
- Review buyer offers and accept, reject, or submit counteroffers.
- Confirm orders and update their status.
- Mark completed products as sold out.
- View demand analysis and buyer feedback.

### 2.2.3. Buyer Functions

- Browse and search available agricultural products.
- Filter products according to district and city.
- View recommended nearby products based on the buyer's location.
- View product, farmer, price, quantity, and location information.
- Submit an offer without going below the farmer's minimum price.
- Monitor offer status and respond to farmer counteroffers.
- Track orders from confirmation to completion.
- Provide ratings and comments for completed orders.

### 2.2.4. Administrator Functions

- View platform statistics through the admin dashboard.
- View registered buyers and farmers.
- Block or reactivate user accounts.
- Monitor product listings.
- Remove inappropriate product listings and restore removed products.
- View administrative activity logs.
- Monitor completed orders and sales information.
- Filter reports and download the Sales and Orders Report as a PDF.

### 2.2.5. Smart System Functions

The system records buyer activities such as product searches, views, submitted offers, and completed orders. This activity information is analysed to classify products as having high or low demand.

The smart pricing mechanism uses relevant completed-order prices and marketplace product prices to calculate a suggested price range. It also considers product demand and ensures that the recommended price does not fall below the farmer's minimum price.

Location-based matching uses the registered district and city information of users and products to display nearby products and support location-based filtering.

## 2.3. Software Requirements

The following software technologies and tools were required to develop and operate the system:

| Software/Technology | Purpose |
|---|---|
| Visual Studio Code | Source-code development |
| PHP 8.3 | Server-side programming |
| Laravel 13 | Web application framework |
| MySQL | Relational database management |
| Laravel Jetstream and Fortify | Authentication and account management |
| Tailwind CSS | Responsive user-interface styling |
| Alpine.js | Interactive frontend behaviour |
| JavaScript | Client-side functionality |
| Vite | Frontend asset building |
| Composer | PHP dependency management |
| Node.js and NPM | Frontend package management |
| Git and GitHub | Version control and source-code hosting |
| Railway | Online system deployment |
| DomPDF | PDF report generation |

***Table 2.3.1: Software Requirements***

## 2.4. Hardware Requirements

### 2.4.1. Development Hardware

- Computer with a 64-bit processor.
- Minimum 4 GB RAM; 8 GB or more recommended.
- At least 10 GB of available storage.
- Keyboard, mouse, and display.
- Reliable internet connection.

### 2.4.2. User Hardware

- Desktop computer, laptop, tablet, or smartphone.
- Internet connection.
- Modern web browser.
- Minimum screen resolution of approximately 1366 × 768 for an optimal desktop experience.

The system uses a responsive user interface, allowing farmers, buyers, and administrators to access the platform using both desktop and mobile devices.

# CHAPTER 3 – SYSTEM DESIGN

System design describes how the identified requirements of the AgroSmart Marketplace were transformed into a functional web-based solution. The design process focused on developing a user-friendly interface, maintaining accurate relationships between data, protecting role-specific functions, and organizing the application logic in a maintainable manner.

The system was designed for three main user roles: Farmer, Buyer, and Administrator. Each role is provided with a separate interface and access to functions relevant to its responsibilities. The overall design consists of the frontend design, database design, and architectural design of the application.

## 3.1. Frontend Design

The frontend of the AgroSmart Marketplace was designed to provide a modern, clear, and responsive user experience.

- Laravel Blade templates were used to create the system interfaces.
- Tailwind CSS was used for layouts, colours, typography, forms, cards, tables, buttons, and responsive design.
- Alpine.js and JavaScript were used for modal windows, confirmation messages, offer actions, smart pricing requests, and review forms.
- A consistent green, white, and neutral colour scheme was applied to represent agriculture, freshness, trust, and simplicity.
- Separate dashboards and navigation menus were provided for farmers, buyers, and administrators.
- Responsive design allows the system to operate on desktop computers, laptops, tablets, and smartphones.

The main user interfaces of the completed AgroSmart Marketplace are described below.

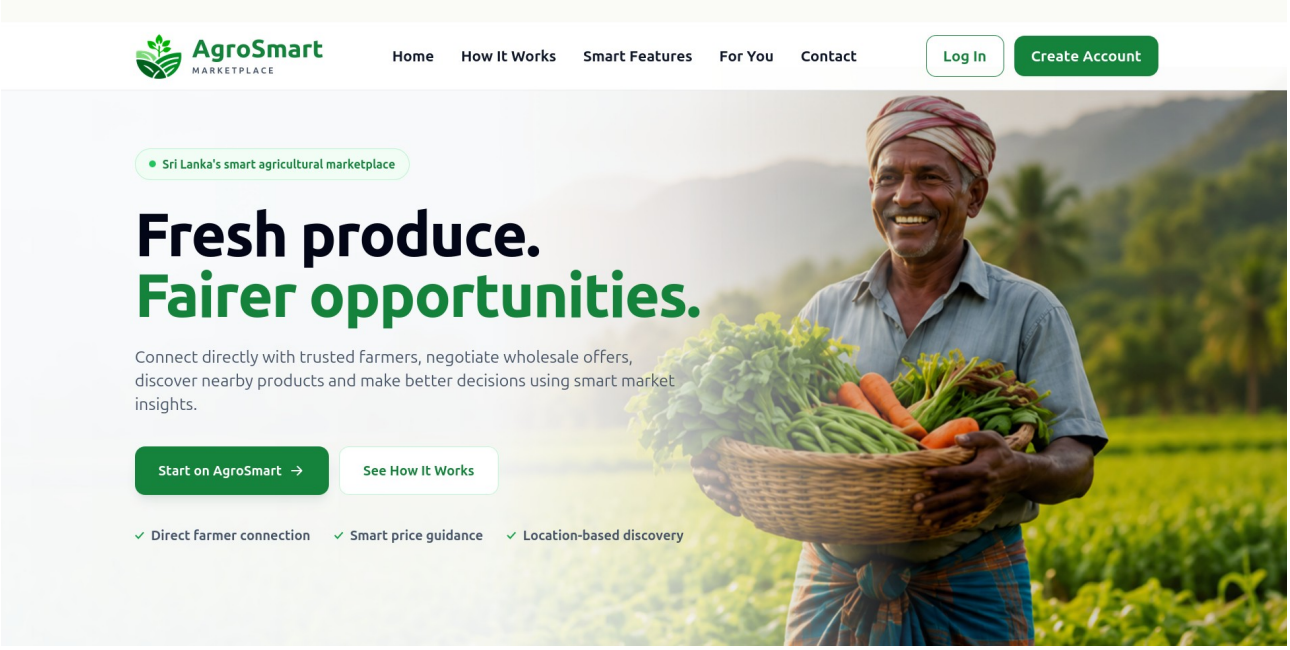



***Figure 3.1.1: Public Landing Page – Hero Section***

The Public Landing Page is the first interface displayed when users access the AgroSmart Marketplace. It introduces the purpose of the platform and highlights key features such as direct product offers, smart pricing, demand analysis, and location-based product discovery. The navigation bar provides access to the main information sections, login page, and registration page. The hero section communicates the platform's objective of creating fairer trading opportunities by connecting farmers and buyers directly. This responsive interface acts as the main entry point for both new and existing users.

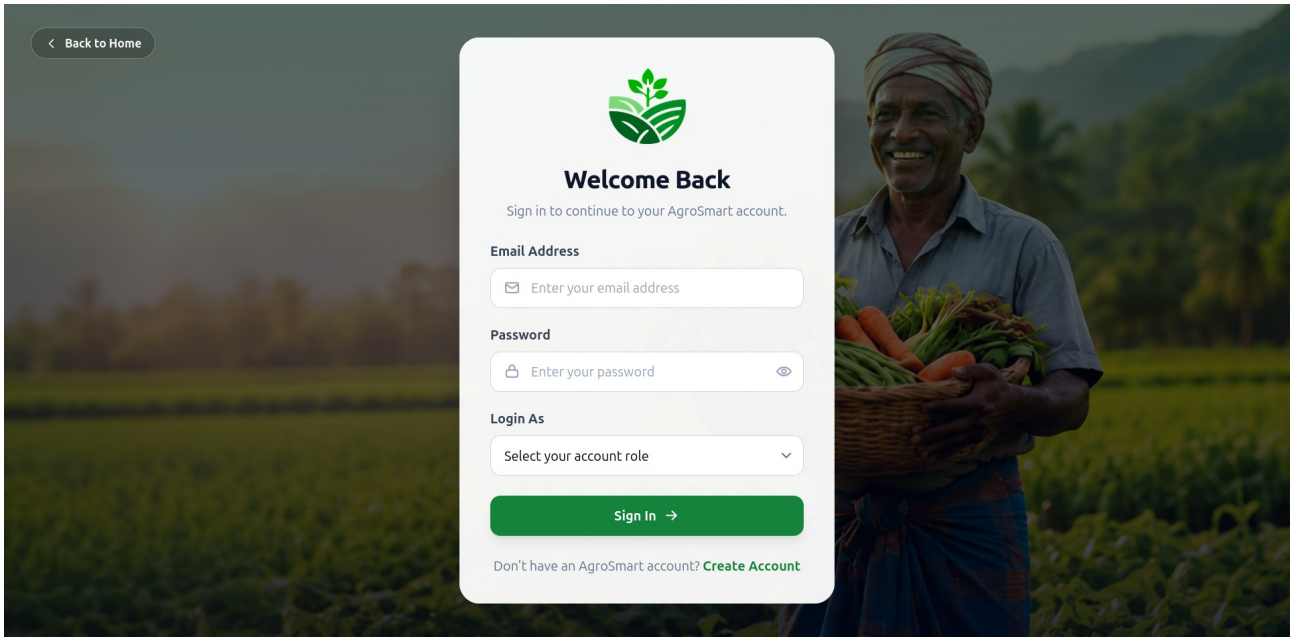



***Figure 3.1.2: Login Interface***

The Login Interface provides secure access to the AgroSmart Marketplace for registered farmers, buyers, and administrators. Users are required to enter their email address, password, and account role before signing in. The system validates the submitted credentials and redirects authenticated users to the dashboard assigned to their role. The interface also provides options for password recovery, new account registration, and returning to the public landing page. Invalid credentials, incorrect roles, and blocked accounts are prevented from accessing protected system functions.

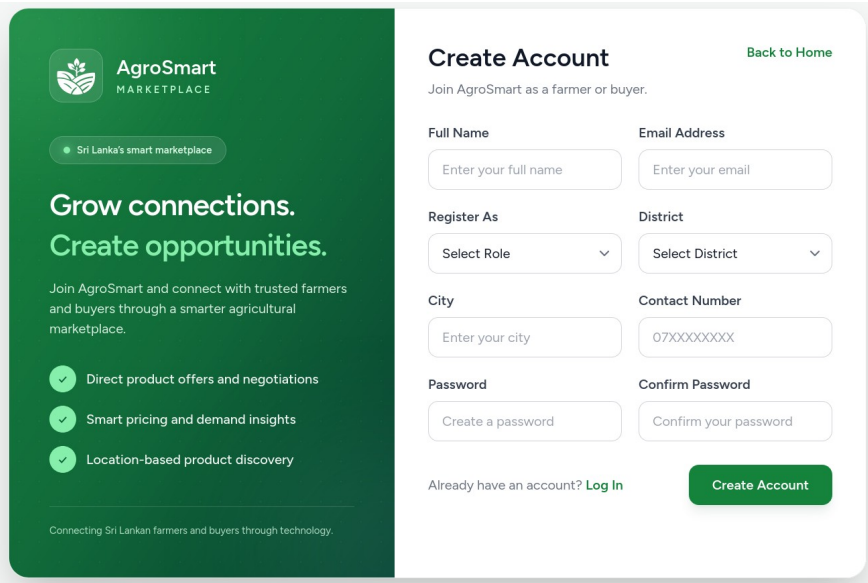



***Figure 3.1.3: User Registration Interface***

The User Registration Interface allows new users to create an AgroSmart account as either a farmer or buyer. The form collects the user's full name, email address, selected role, district, city, contact number, password, and password confirmation. Required fields and validation rules help ensure that accurate information is submitted. The registered location details are later used to provide location-based product filters and nearby product recommendations. After successful registration, the user can proceed to the login interface and access the dashboard assigned to the selected role.

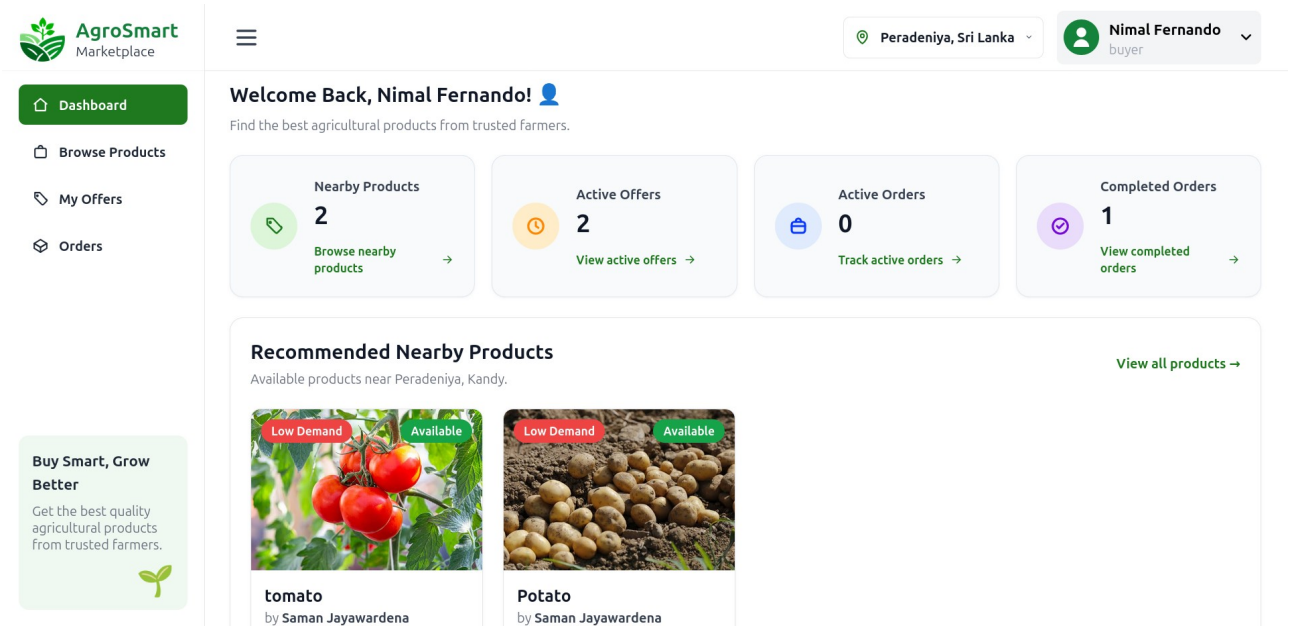



***Figure 3.1.4: Buyer Dashboard Interface***

The Buyer Dashboard is the main interface displayed after a buyer successfully logs in. It provides a summary of nearby products, active offers, active orders, and completed orders through dynamic statistical cards. The dashboard also displays recommended agricultural products based on the buyer's registered district and city, allowing nearby products to be discovered more conveniently. Navigation links provide access to product browsing, offer management, order tracking, and demand insights. The displayed values are retrieved dynamically from the database according to the authenticated buyer's activities and transactions.

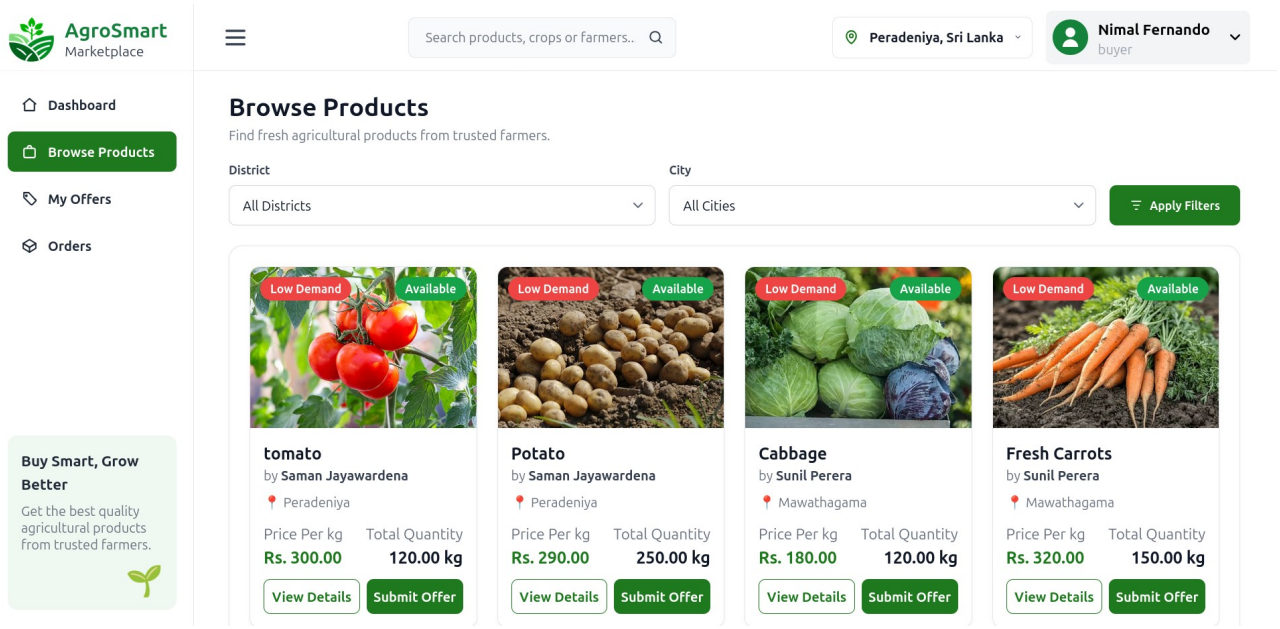



***Figure 3.1.5: Product Browsing***

The Product Browsing Interface allows buyers to discover available agricultural products published by farmers. Buyers can search using product names, categories, farmer names, or locations and filter the results according to district and city. Each product card presents important information such as the product name, farmer, location, selling price, minimum price, available quantity, demand level, and availability status. Products removed by administrators or marked as sold out are restricted from new offer submissions. The location filters and nearby recommendations make it easier for buyers to identify suitable products available closer to their registered location.

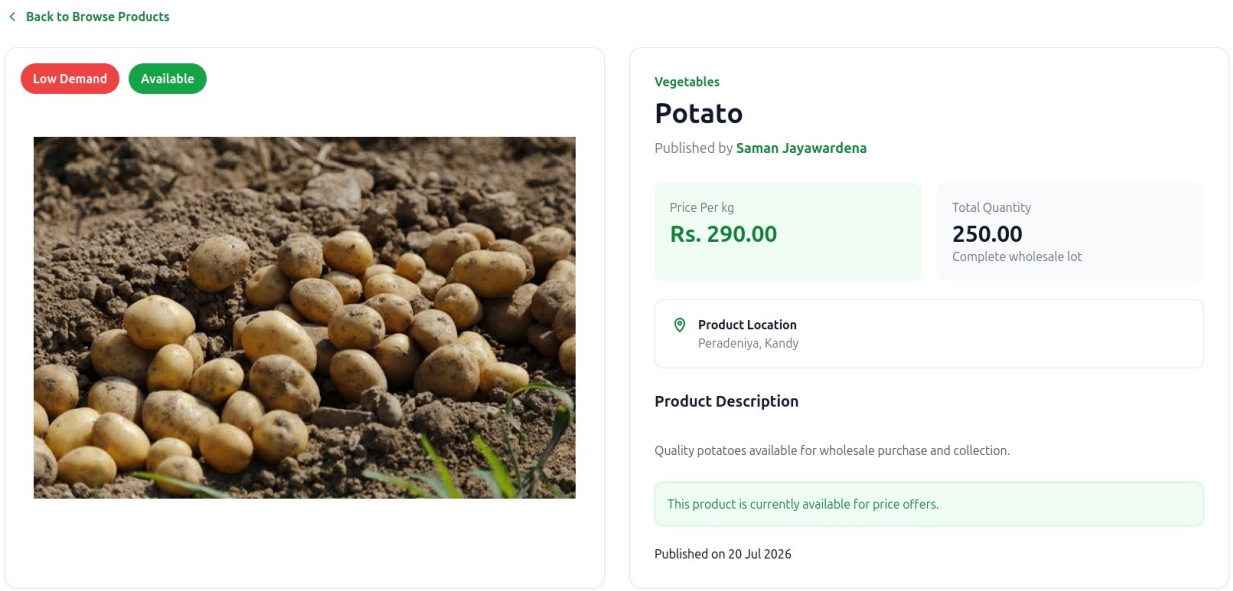



*Figure 3.1.6: Product Details*

The Product Details Interface provides buyers with complete information about a selected agricultural product, including its image, product name, category, farmer, location, quantity, selling price, minimum price, description, demand level, and availability status.

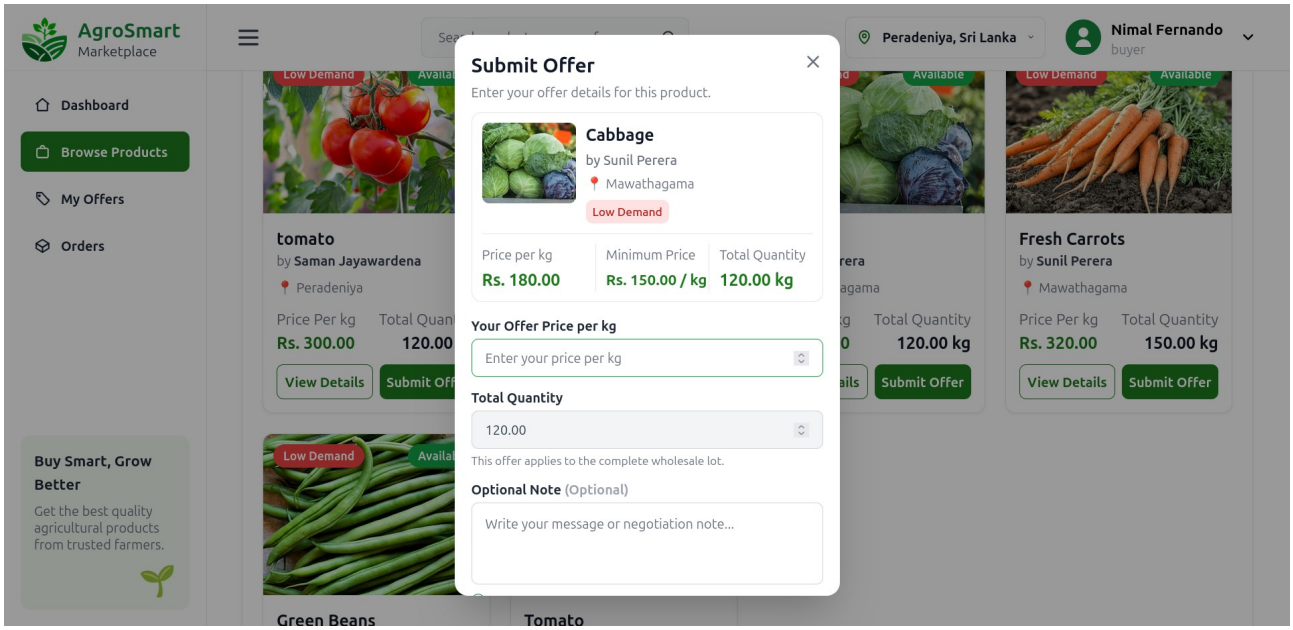



*Figure 3.1.7: Offer Submission Interface*

The offer submission form allows the buyer to enter an offered price, required quantity, and an optional note for the farmer. The system validates the submitted information and prevents offers below the farmer's minimum price, quantities greater than the available stock, duplicate offers for the same product, and offers for sold-out or removed products. After successful submission, the offer is recorded with a pending status for the farmer to review.

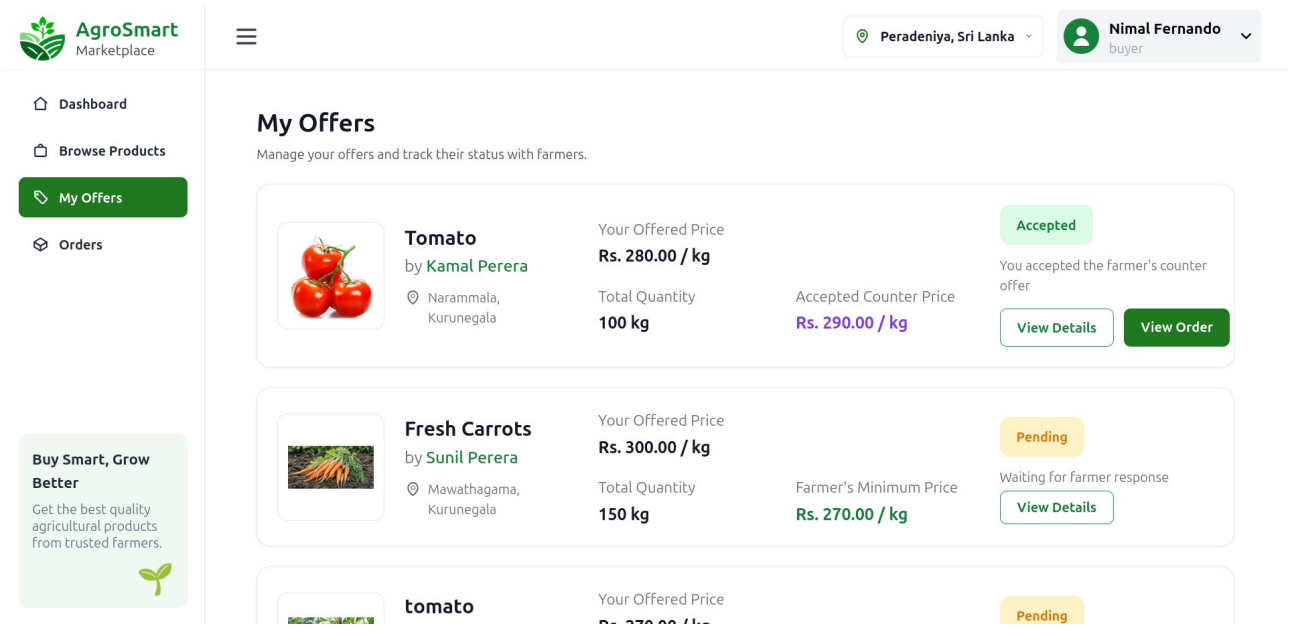



***Figure 3.1.8: Buyer Offer Management Interface***

The Buyer Offer Management Interface allows buyers to monitor all submitted product offers and their current negotiation status. Each offer displays the product, farmer, location, offered price, requested quantity, minimum price, counter price, and submission date. Status badges clearly identify pending, countered, accepted, and rejected offers. When a farmer submits a counteroffer, the buyer can review the revised price and note before accepting or rejecting it. Accepted offers proceed to order creation, while rejected offers remain available as transaction history. This interface provides buyers with a clear and structured view of the complete price-negotiation process.

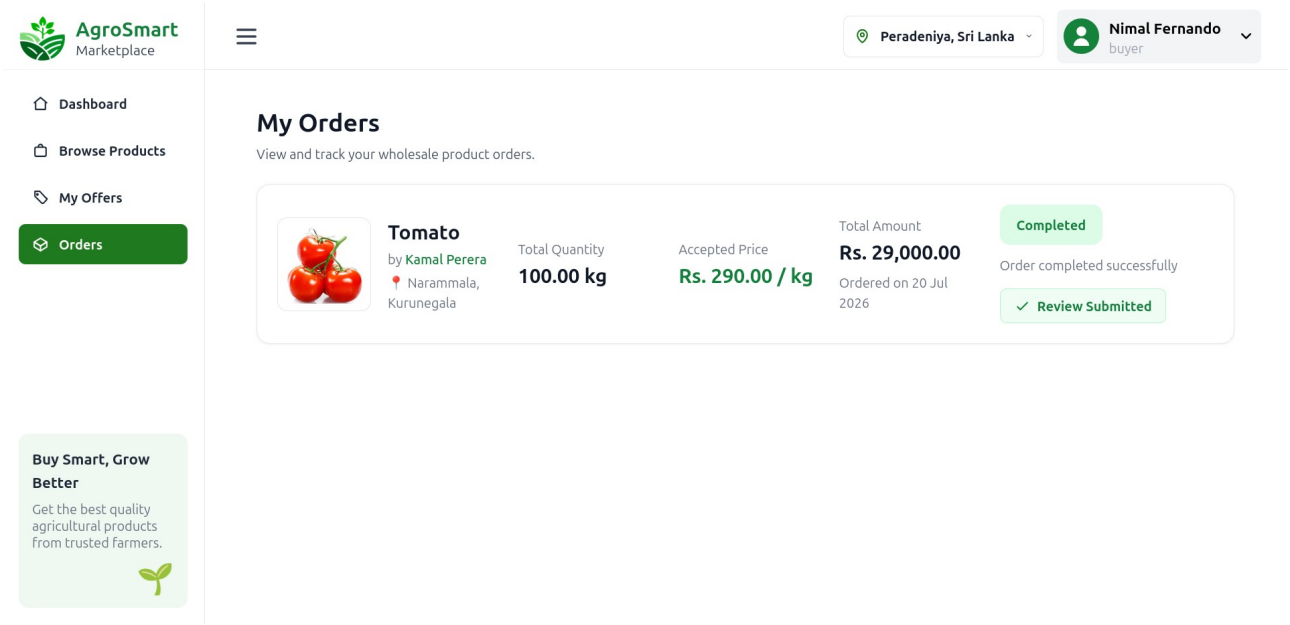



***Figure 3.1.9: Buyer Order Tracking and Review Interface***

The Buyer Order Tracking Interface allows buyers to monitor orders created from accepted product offers. Each order displays the product, farmer, location, ordered quantity, accepted price, total amount, order date, and current status. Status indicators show whether an order is confirmed, ready for pickup, completed, or cancelled. After an order has been completed, the buyer can use the review function to submit a rating and comment about the transaction. The system connects the feedback to the relevant order, product, buyer, and farmer while preventing duplicate reviews for the same order.

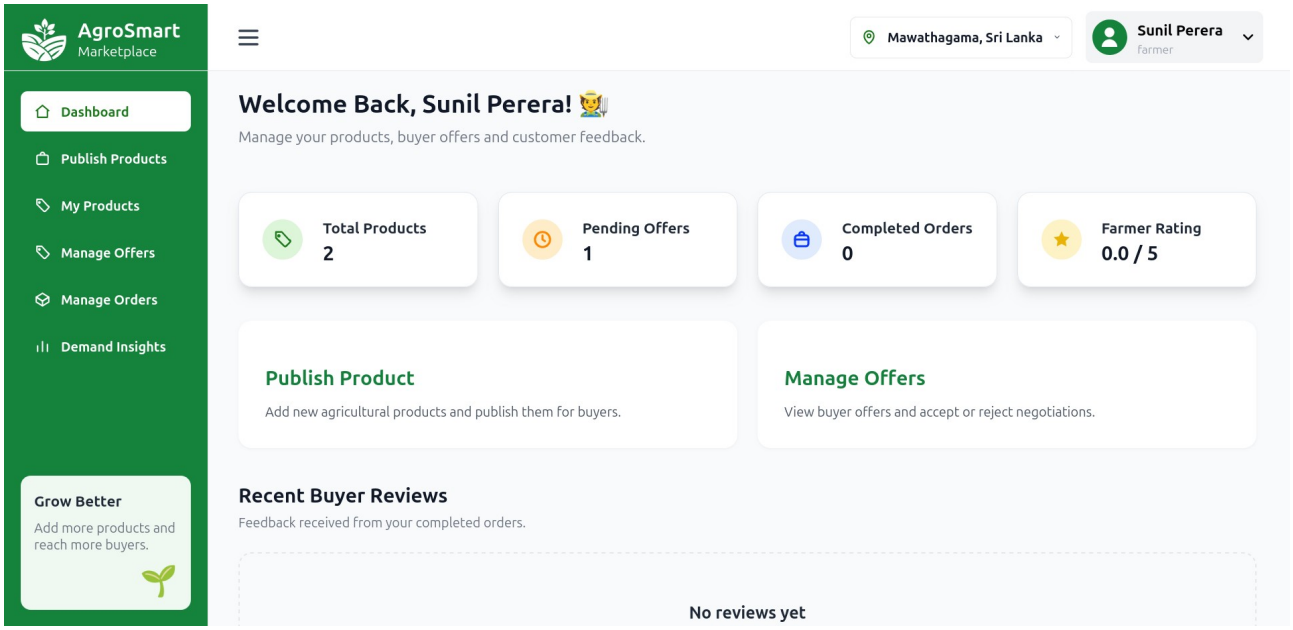



***Figure 3.1.10: Farmer Dashboard Interface***

The Farmer Dashboard is the main interface displayed after a farmer successfully logs in. It provides a summary of published products, pending offers, confirmed orders, completed orders, and other relevant marketplace activities. Dynamic cards and recent-record sections allow the farmer to quickly understand current product and transaction information. Navigation links provide access to product publishing, product management, buyer offers, order confirmation, and demand analysis. The displayed information is retrieved from the database according to the authenticated farmer's products, offers, and orders, providing a centralized view of agricultural trading activities.

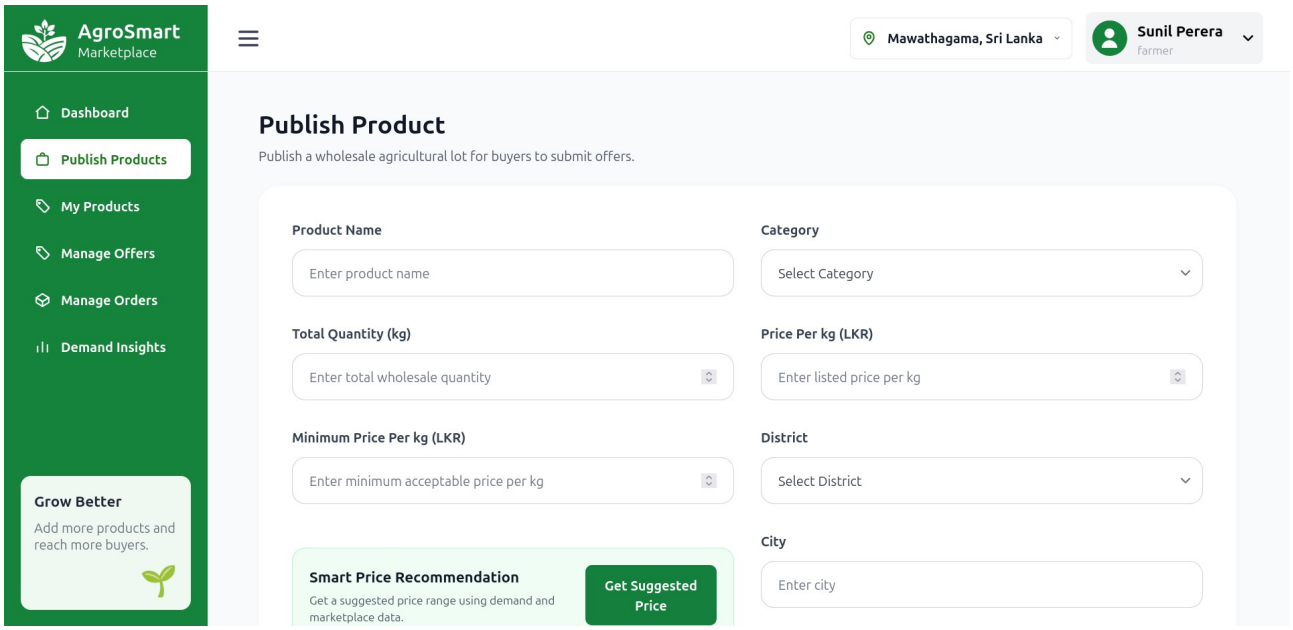



***Figure 3.1.11: Product Publishing and Smart Pricing Interface***

The Product Publishing Interface allows farmers to add new agricultural products to the AgroSmart Marketplace. The form collects the product name, category, total quantity, selling price, minimum acceptable price, district, city, product image, and description. Validation rules ensure that the submitted values are complete and correctly formatted before the product is stored.

The Smart Price Recommendation component assists the farmer in selecting a suitable selling price. It analyses relevant completed-order prices, existing marketplace prices, product demand, category, and location to calculate a suggested price and price range. The recommendation is prevented from falling below the farmer's minimum price. The farmer can review the result and apply the suggested price directly to the product form before publishing the listing.

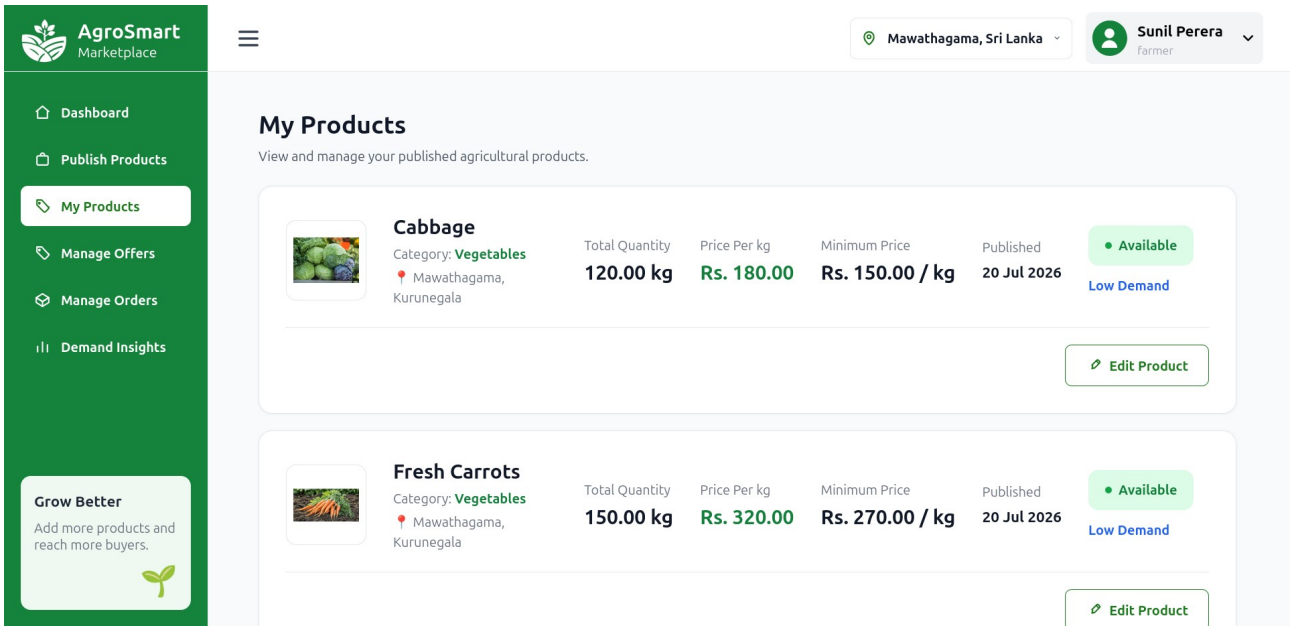



***Figure 3.1.12: Farmer Product Management Interface***

The Farmer Product Management Interface allows farmers to view and manage all agricultural products published through their accounts. Each product record displays the product image, name, category, price, minimum price, available quantity, location, demand level, availability, and publication date. Farmers can select active products and update their information when necessary. Products that have been fully sold are marked as sold out, preventing further offer submissions and inappropriate modifications. This interface provides farmers with a structured view of their product inventory and current marketplace status.

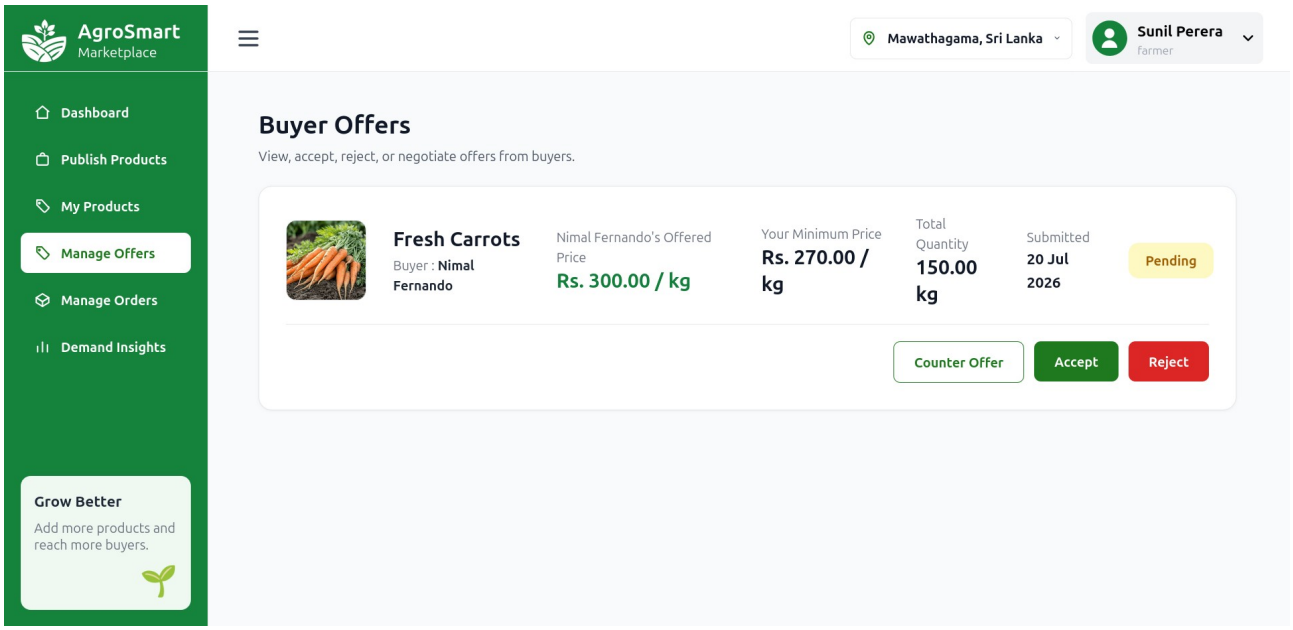



***Figure 3.1.13: Farmer Offer Management Interface***

The Farmer Offer Management Interface allows farmers to review and respond to offers submitted by buyers. Each offer displays the product, buyer, offered price, farmer's minimum price, requested quantity, submission date, and current negotiation status. A farmer can accept a suitable offer, reject an unsuitable offer, or submit a counteroffer with a revised price and optional note.

The system records whether an offer was accepted or rejected by the farmer or buyer and updates its status accordingly. When an offer is accepted, an order is created using the agreed price and quantity, while competing offers for the sold product can be closed appropriately. This interface provides a structured and transparent price-negotiation process between farmers and buyers.

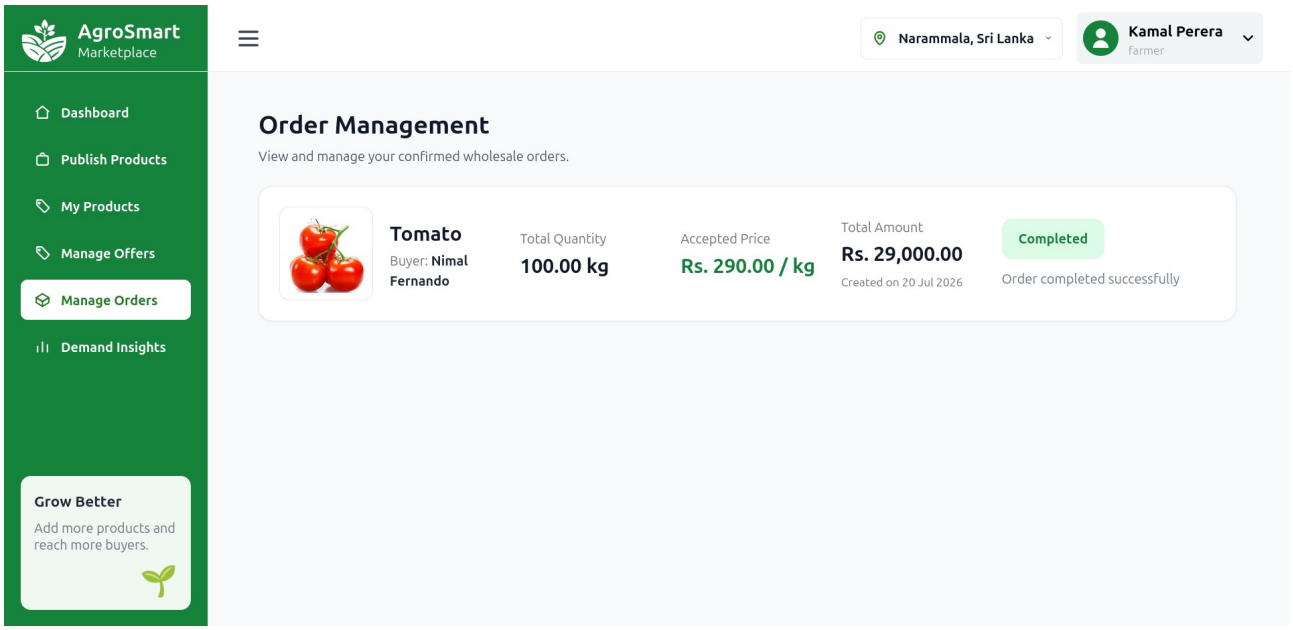



***Figure 3.1.14: Farmer Order Management Interface***

The Farmer Order Management Interface displays orders created from accepted buyer offers. Each order contains the product, buyer, agreed quantity, accepted price, total amount, order date, and current status. Farmers can manage the transaction by updating the order through the permitted stages, including confirmed, ready for pickup, and completed. An order may also be cancelled where allowed by the order workflow.

The system validates each status transition to prevent invalid changes. When an order is completed, the related product is marked as sold out and the completed transaction is recorded for demand analysis and sales reporting. If an order is cancelled, the product can be made available again according to the transaction rules. This interface enables farmers to manage orders through a clear and controlled process.

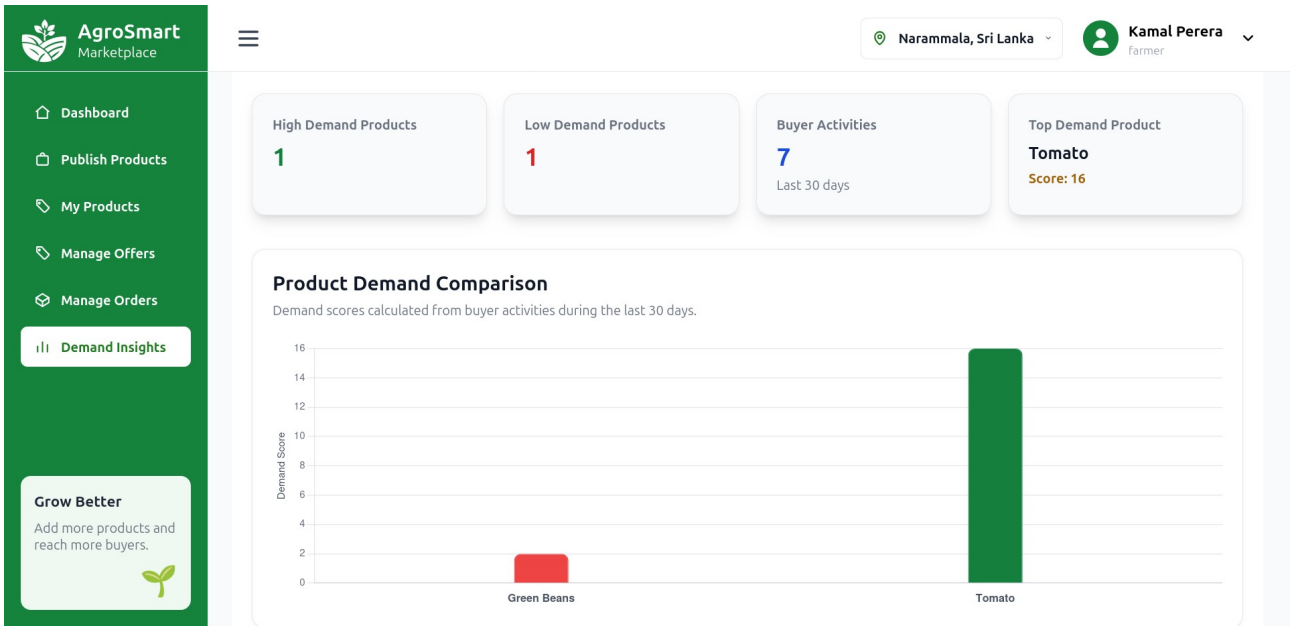



*Figure 3.1.15: Demand Analysis Interface*

The Demand Analysis Interface helps farmers understand buyer interest and market demand for their agricultural products. The system records activities such as product searches, product views, submitted offers, and completed orders. These activities are assigned demand values and analysed to determine whether each product has high or low demand.

The interface presents demand information through summary cards, product records, and visual charts, allowing farmers to compare product performance. Demand levels are recalculated when relevant buyer activities are recorded, providing updated marketplace insights. Farmers can use this information together with smart pricing recommendations to make informed decisions about product availability, pricing, and future agricultural production.

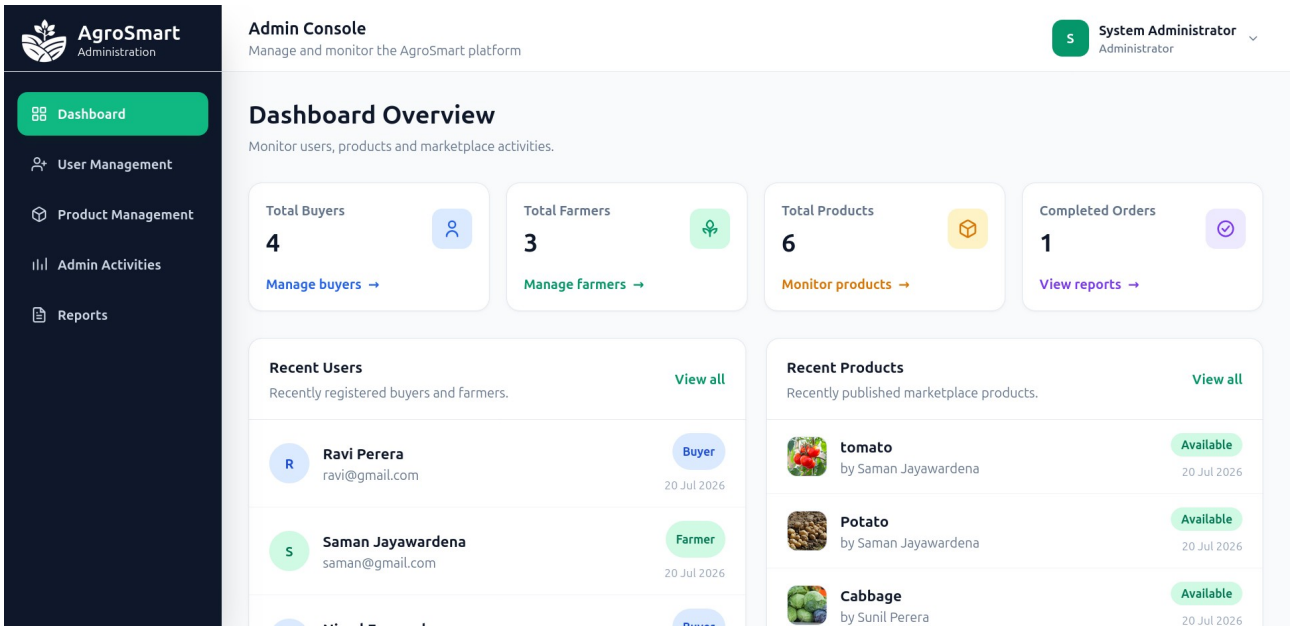



***Figure 3.1.16: Administrator Dashboard Interface***

The Administrator Dashboard is the main monitoring interface available to authorized administrators. It provides a dynamic overview of the total number of buyers, farmers, products, and completed orders within the AgroSmart Marketplace. Quick-access links allow administrators to navigate directly to user management, product monitoring, administrative activity logs, and sales reports.

The dashboard also displays recently registered users and recently published agricultural products, including their roles, publication dates, farmers, and availability status. All statistics and recent records are retrieved dynamically from the database. This interface enables administrators to quickly evaluate current marketplace activity and access the platform's main management functions from a centralized location.

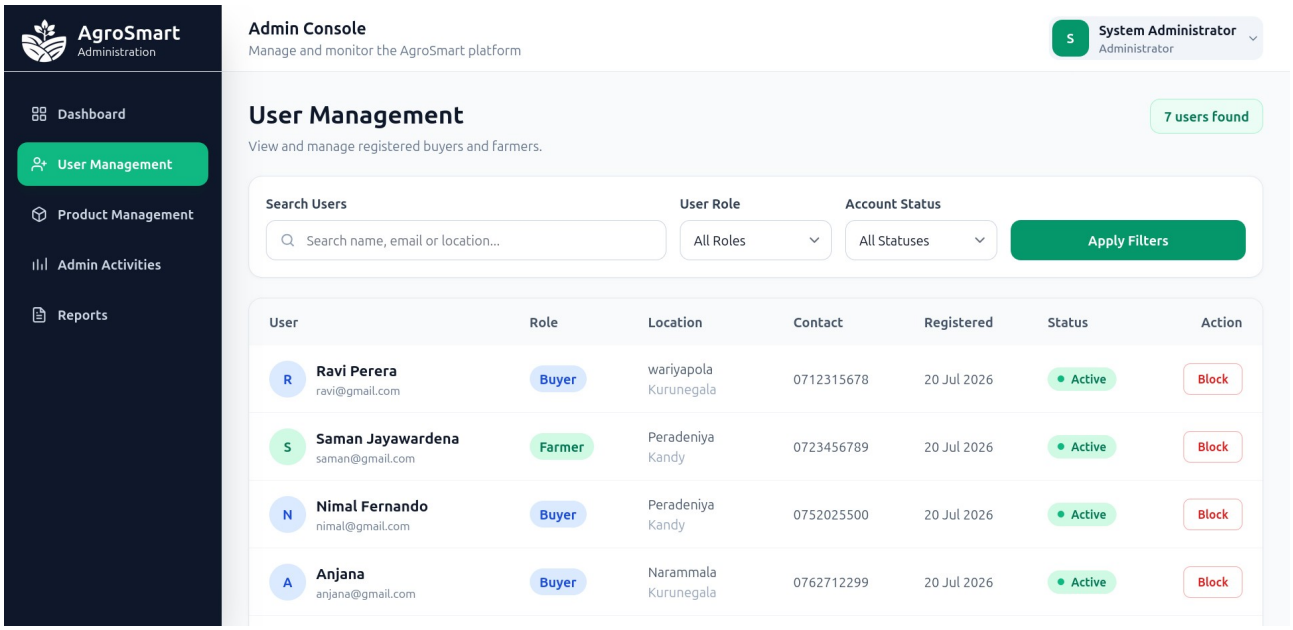



***Figure 3.1.17: User Management Interface***

The User Management Interface allows administrators to view and manage registered buyer and farmer accounts. The interface displays each user's name, email address, role, location, contact number, registration date, and current account status. Search and filter controls enable administrators to locate users by name, email address, role, location, or account status.

Administrators can block accounts that violate marketplace requirements and reactivate previously blocked accounts when necessary. Blocked users are prevented from logging in or accessing protected system functions through account-status middleware. Administrator accounts are excluded from these actions to reduce the risk of unauthorized administrative changes. Each account-status update is also recorded in the administrative activity log for accountability and future monitoring.

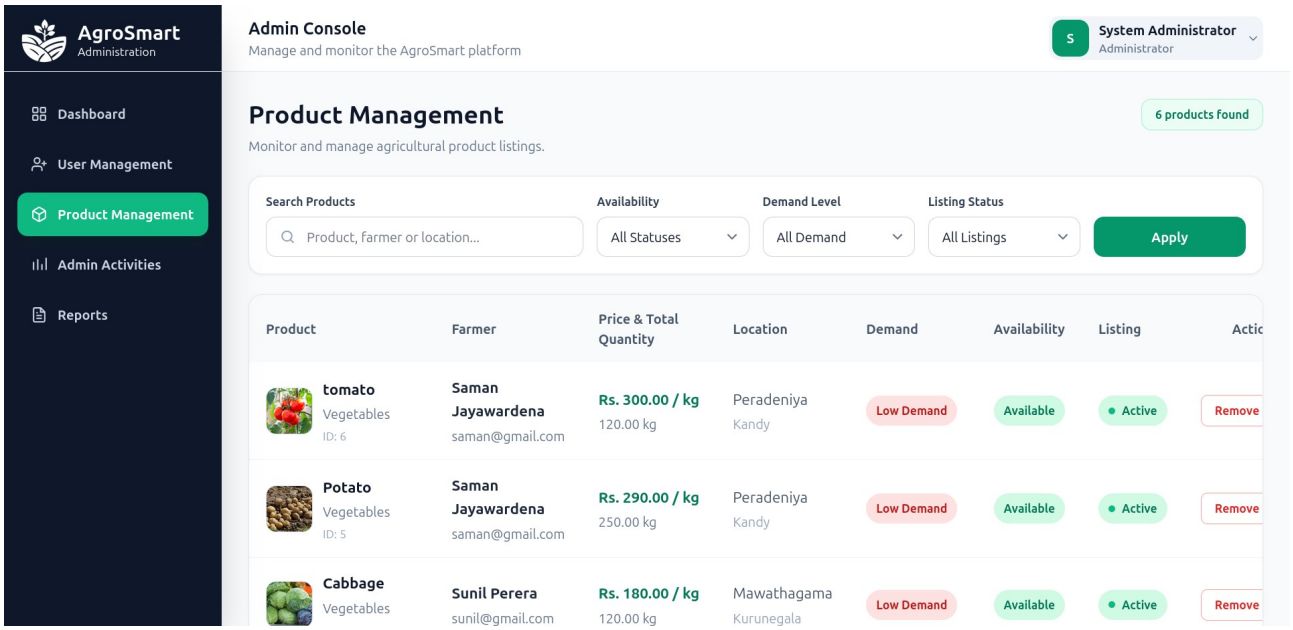



***Figure 3.1.18: Product Management Interface***

The Product Management Interface allows administrators to monitor agricultural products published by farmers. Each listing displays the product name, category, farmer details, price, quantity, location, demand level, availability, and moderation status. Search and filter controls enable products to be identified according to product information, farmer, availability, demand level, or listing status.

Administrators can remove inappropriate, misleading, or invalid product listings by providing a removal reason. Removed products are hidden from buyers but remain stored in the database for monitoring and possible restoration. If a listing is reviewed and considered acceptable, the administrator can restore it to the active marketplace. Product removal and restoration actions are recorded in the administrative activity log, including the responsible administrator and action details.

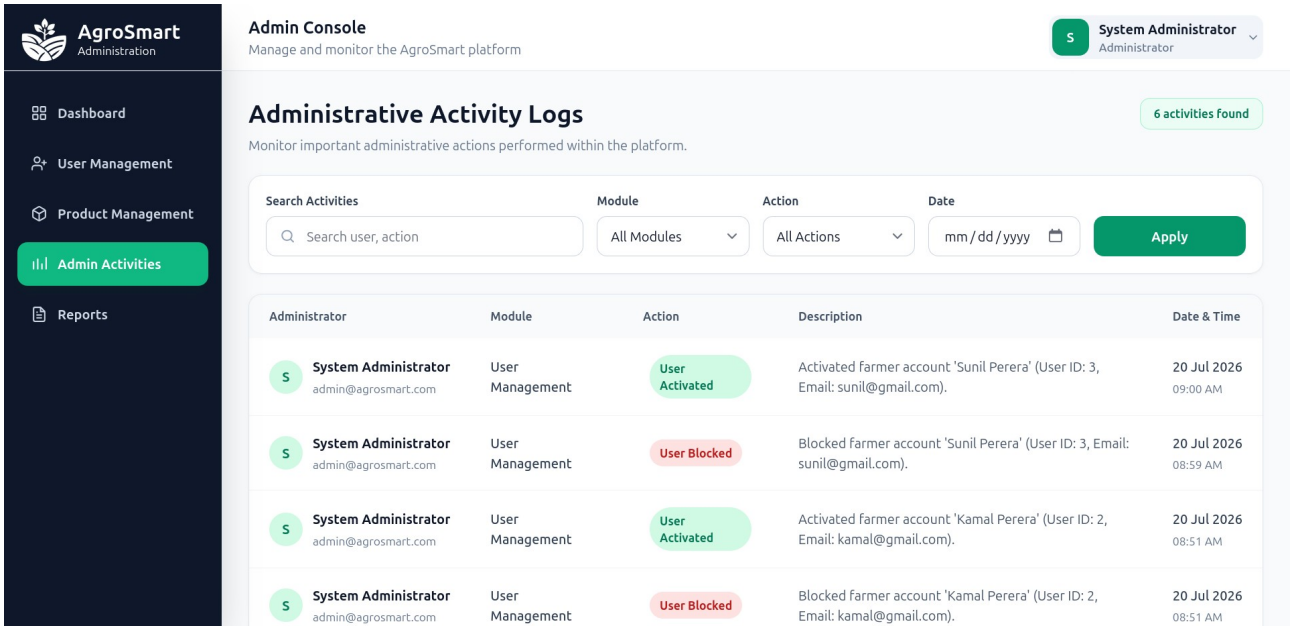



***Figure 3.1.19: Administrative Activity Logs Interface***

The Administrative Activity Logs Interface provides a record of important management actions performed within the AgroSmart Marketplace. Each activity includes the administrator's identity, related module, action type, description, IP address, and date and time. Recorded actions include blocking or activating user accounts and removing or restoring product listings.

Administrators can search activity records and filter them according to module, action, or date. The records are arranged in chronological order to make recent administrative changes easy to identify. This interface improves accountability and platform security by providing a traceable history of significant administrative actions and the administrator responsible for each action.

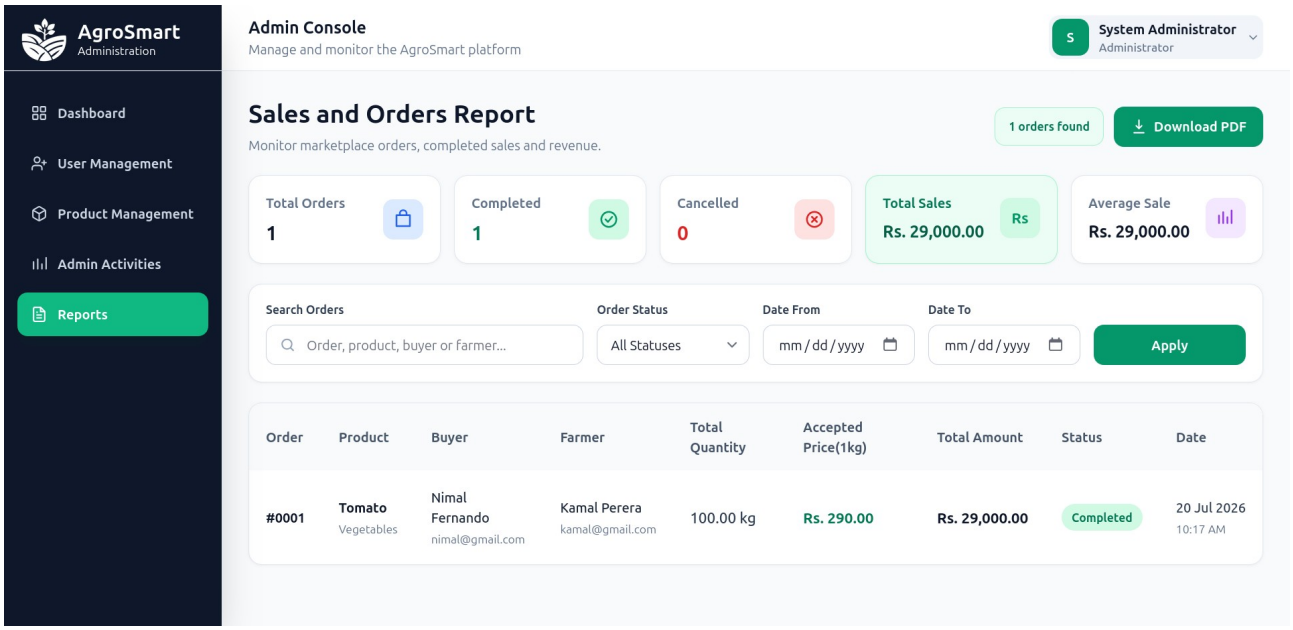



***Figure 3.1.20: Sales and Orders Report Interface***

The Sales and Orders Report Interface allows administrators to monitor completed marketplace transactions and total sales information. The report displays order, product, buyer, farmer, quantity, accepted price, total amount, order status, and transaction date. Summary information provides an overview of completed orders and their combined sales value.

Administrators can filter the report using relevant criteria such as date range, order status, product, buyer, or farmer. The displayed information is retrieved dynamically from the orders and related database tables. A Download PDF function generates a formatted report containing the selected transaction information, allowing it to be stored, printed, or shared for administrative and academic purposes.

## 3.2. Database Design

The database of the AgroSmart Marketplace was designed using MySQL to store and manage information related to users, products, offers, orders, reviews, demand activities, and administrative activities. The database uses primary keys to uniquely identify records and foreign keys to establish relationships between related tables. This relational structure supports data consistency and the complete transaction workflow between farmers, buyers, and administrators.

The main database tables are described below.

### 3.2.1. Database Tables

**1. Users Table**

The **users** table stores the account information of all users registered in the AgroSmart Marketplace. It contains authentication details, personal information, location details, contact information, user role, and account status. The system supports three main user roles: Buyer, Farmer, and Administrator. This table is also used to identify the user responsible for products, offers, orders, reviews, demand activities, and administrative activities.

**Relationships:**

- One Farmer can publish many Products.
- One Buyer can submit many Offers.
- One Buyer can place many Orders.
- One Farmer can receive many Orders.
- One Buyer can submit many Reviews.
- One Farmer can receive many Reviews.
- One Buyer can generate many Demand Activity records.
- One Administrator can generate many System Activity records

**2. Products Table**

The **products** table stores agricultural products published by farmers. It contains information such as the product name, category, available quantity, measurement unit, selling price, minimum acceptable price, product location, description, image, demand level, and availability status. Each product is linked to the farmer who published it through the `farmer_id` foreign key.

**Relationships:**

- Each Product belongs to one Farmer.
- One Farmer can publish many Products.
- One Product can receive many Offers.
- One Product can be associated with many Orders.
- One Product can receive many Reviews.
- One Product can have many Demand Activity records.
- One Product may be removed or restored by an Administrator.

**3. Offers Table**

The **offers** table stores price offers submitted by buyers for agricultural products. It records the buyer's offered price, requested quantity, optional note, farmer counter price, counter note, negotiation status, and the person who accepted or rejected the offer. This table supports the offer negotiation process between buyers and farmers before an order is created.

**Relationships:**

- Each Offer belongs to one Buyer.
- One Buyer can submit many Offers.
- Each Offer is related to one Product.
- One Product can receive many Offers.
- One accepted Offer can generate zero or one Order.
- Each Order created through negotiation belongs to one accepted Offer.

### 4. Orders Table

The **orders** table stores confirmed transactions created after an offer has been accepted. It contains the buyer, farmer, product, accepted price, ordered quantity, total amount, and current order status. The order status is used to track the transaction through stages such as pending, confirmed, ready for pickup, completed, or cancelled.

**Relationships:**

- Each Order belongs to one Buyer.
- One Buyer can place many Orders.
- Each Order belongs to one Farmer.
- One Farmer can receive many Orders.
- Each Order is associated with one Product.
- One Product can be associated with many Orders.
- Each Order is created from one accepted Offer.
- One completed Order can have zero or one Review.

### 5. Reviews Table

The **reviews** table stores ratings and feedback submitted by buyers after completing an order. It contains the rating value, optional review comment, buyer information, and references to the related order, product, and farmer. Reviews allow buyers to share their experience and help improve trust between users in the marketplace.

**Relationships:**

- Each Review belongs to one completed Order.
- One Order can have zero or one Review.
- Each Review is submitted by one Buyer.
- One Buyer can submit many Reviews.
- Each Review is associated with one Farmer.
- One Farmer can receive many Reviews.
- Each Review is related to one Product.
- One Product can have many Reviews.

### 6. Demand Activities Table

The **demand_activities** table stores buyer activities used to analyze product demand. These activities may include product searches, product views, submitted offers, and completed orders. The collected activity information is used to identify high-demand and low-demand products and to provide useful demand insights within the system.

**Relationships:**

- Each Demand Activity belongs to one Buyer.
- One Buyer can generate many Demand Activity records.
- Each Demand Activity is related to one Product.
- One Product can have many Demand Activity records.
- Demand Activity records are used to calculate and update the demand level of Products.

### 7. System Activities Table

The **`system_activities`** table records important administrative actions performed within the AgroSmart Marketplace. It provides an audit trail for user management, product moderation, and other administrative operations. The table may store the action name, affected system module, detailed description, IP address, browser or device information, and the date and time the activity occurred.

Examples of recorded actions include user blocking, user activation, product removal, and product restoration. These records help administrators monitor system operations and maintain accountability.

**Relationships:**

- Each System Activity belongs to one Administrator.
- One Administrator can generate many System Activity records.
- One Administrator can block or activate multiple user accounts.
- One Administrator can remove or restore multiple product listings.
- The **`user_id`** foreign key references the administrator in the **`users`** table.

### 3.2.2. Database Relationship

| Relationship | Cardinality | Description |
|---|---|---|
| Farmer → Products | 1 : M | One farmer can publish many products, while each product belongs to one farmer. |
| Buyer → Offers | 1 : M | One buyer can submit many offers, while each offer belongs to one buyer. |
| Product → Offers | 1 : M | One product can receive many offers from buyers. |
| Offer → Order | 1 : 0..1 | An accepted offer can generate one order, while each order is created from one offer. |
| Buyer → Orders | 1 : M | One buyer can place many orders, while each order belongs to one buyer. |
| Farmer → Orders | 1 : M | One farmer can receive many orders, while each order belongs to one farmer. |
| Product → Orders | 1 : M | One product can be associated with multiple orders. |
| Order → Review | 1 : 0..1 | A completed order can receive one review, while each review belongs to one order. |
| Buyer → Reviews | 1 : M | One buyer can submit many reviews. |
| Farmer → Reviews | 1 : M | One farmer can receive many reviews from buyers. |
| Product → Reviews | 1 : M | One product can be associated with multiple reviews. |
| Buyer → Demand Activities | 1 : M | One buyer can generate many search, view, offer, and order activities. |
| Product → Demand Activities | 1 : M | One product can have many recorded demand activities. |
| Administrator → System Activities | 1 : M | One administrator can generate many administrative activity records. |
| Administrator → Removed Products | 1 : M | One administrator can remove multiple product listings. |

***Table 3.2.2.1: Database Relationship***

### 3.2.3. Entity Relationship Diagram

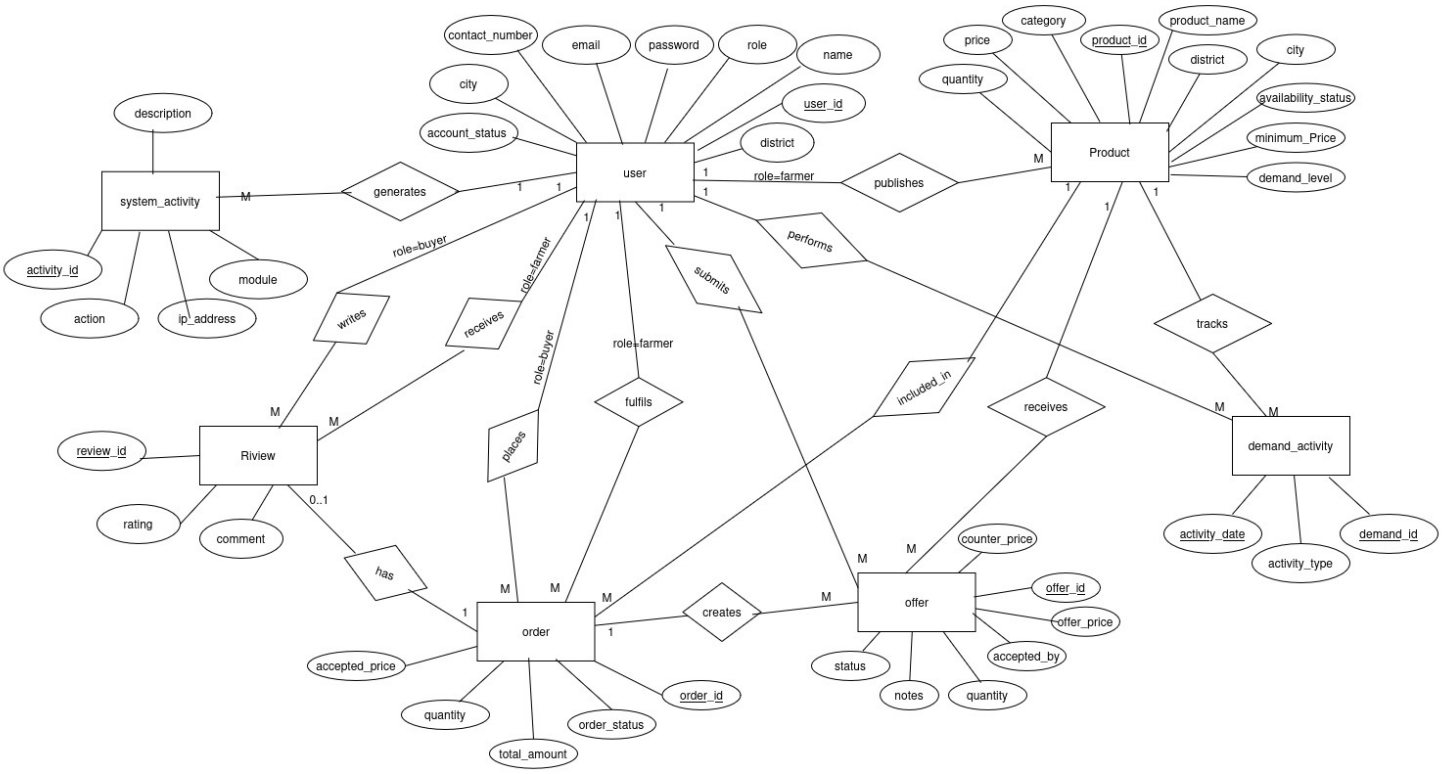



*Figure 3.2.3.1: Entity Relationship Diagram*

## 3.3. Architectural Design

### 3.3.1. Model–View–Controller (MVC)

The AgroSmart Marketplace follows the **Model–View–Controller (MVC)** architectural pattern provided by the Laravel framework. This architecture separates the system's user interfaces, business logic, and database operations, making the application easier to develop, maintain, test, and extend.

1. **Model**

The Model manages the system's data and database operations.

- It represents MySQL database tables through Laravel Eloquent ORM.
- It defines relationships between entities, such as users, products, offers, orders, and reviews.
- It performs data creation, retrieval, updating, and deletion.
- The main models are **User, Product, Offer, Order, Review, DemandActivity,** and **SystemActivity.**

**2. View**

The View represents the user interface of the system.

- It displays system information to farmers, buyers, and administrators.
- Laravel Blade templates are used to develop the web pages.
- Tailwind CSS provides a modern and responsive interface.
- Alpine.js and JavaScript support interactive components such as forms, menus, and modal windows.
- Examples include dashboards, product listings, offer pages, order pages, and administration pages.

**3. Controller**

The Controller acts as the connection between the Model and the View.

- It receives user requests through Laravel routes.
- It validates user input and applies the system's business rules.
- It communicates with Models to retrieve or modify database records.
- It sends processed data to the relevant View.
- It manages functions such as product publishing, offer negotiation, order processing, review submission, and administrative actions.

**MVC Request Flow**

1. The user performs an action through a View.
2. A Laravel route sends the request to the relevant Controller.
3. Middleware verifies authentication and role-based access.
4. The Controller validates the request and executes the business logic.
5. The Controller uses a Model to access the MySQL database.
6. The Model retrieves or updates the required data.
7. The Controller passes the processed data to a View.
8. The View displays the final response to the user.

### 3.3.2. Three-Tier Architecture

The system also follows a **three-tier architecture**, consisting of the presentation layer, application layer, and data layer.

**1. Presentation Layer**

The presentation layer provides the interfaces used by farmers, buyers, and administrators. Laravel Blade templates, Tailwind CSS, Alpine.js, and JavaScript are used to create responsive and interactive web pages.

The main interfaces include:

- User registration and login
- Farmer dashboard and product management
- Buyer dashboard and product browsing
- Offer negotiation interfaces
- Order management interfaces
- Review and feedback interfaces
- Administrator dashboard and moderation interfaces

**2. Application Layer**

The application layer processes user requests and contains the main business logic of the system. Laravel routes direct incoming requests to the appropriate controllers.

The controllers handle operations such as:

- Authenticating users and applying role-based access
- Publishing and updating products
- Submitting, accepting, rejecting, and countering offers
- Generating orders from accepted offers
- Updating order statuses
- Submitting ratings and reviews
- Analysing product demand
- Managing users and moderating product listings
- Recording administrative activities

Laravel middleware protects system functions and ensures that only authorized users can access role-specific features.

**3. Data Layer**

The data layer uses Eloquent ORM to communicate with the MySQL database. Models represent the system's main entities and manage their database relationships.

The principal models are:

- User
- Product
- Offer
- Order
- Review
- DemandActivity
- SystemActivity

The database stores user information, product listings, offer negotiations, confirmed orders, buyer reviews, demand-related activities, and administrative audit records.

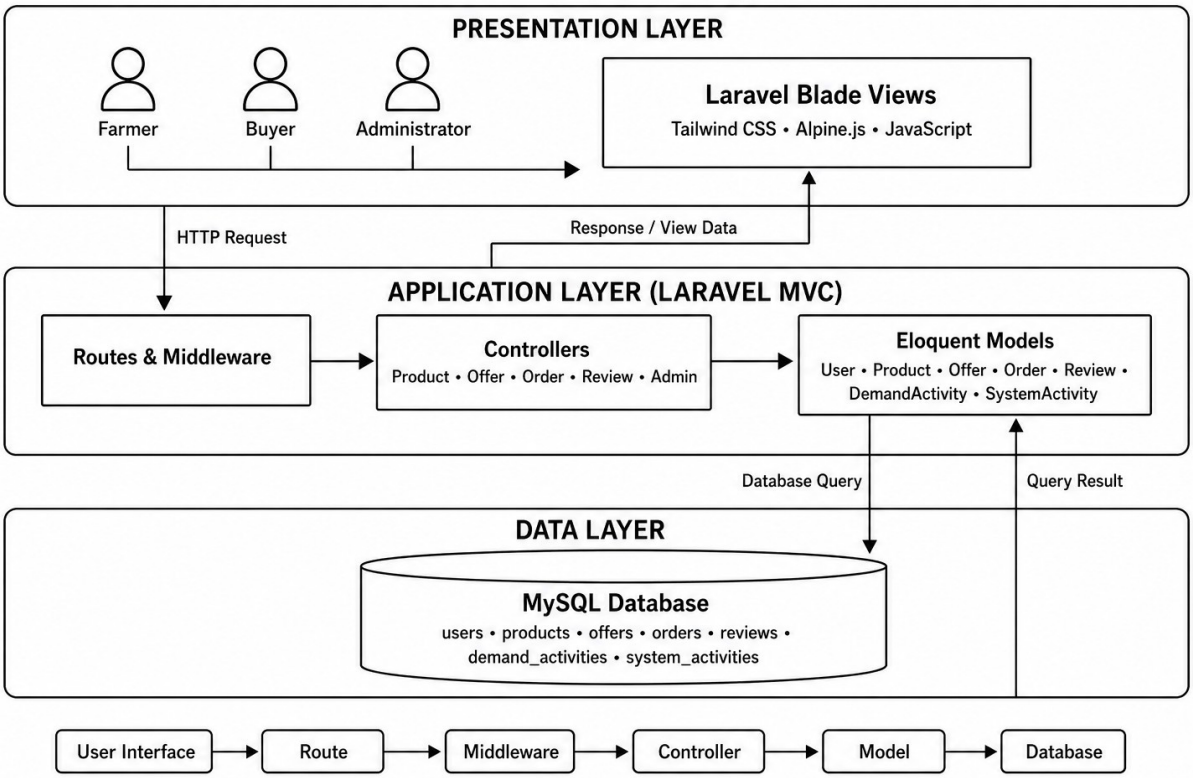



*Figure 3.3.2.1: MVC and Three-Tier Architecture*

## 3.4. Use Case Diagram




***Figure 3.4.1: Use Case Diagram***

## 3.5. UML Class Diagram

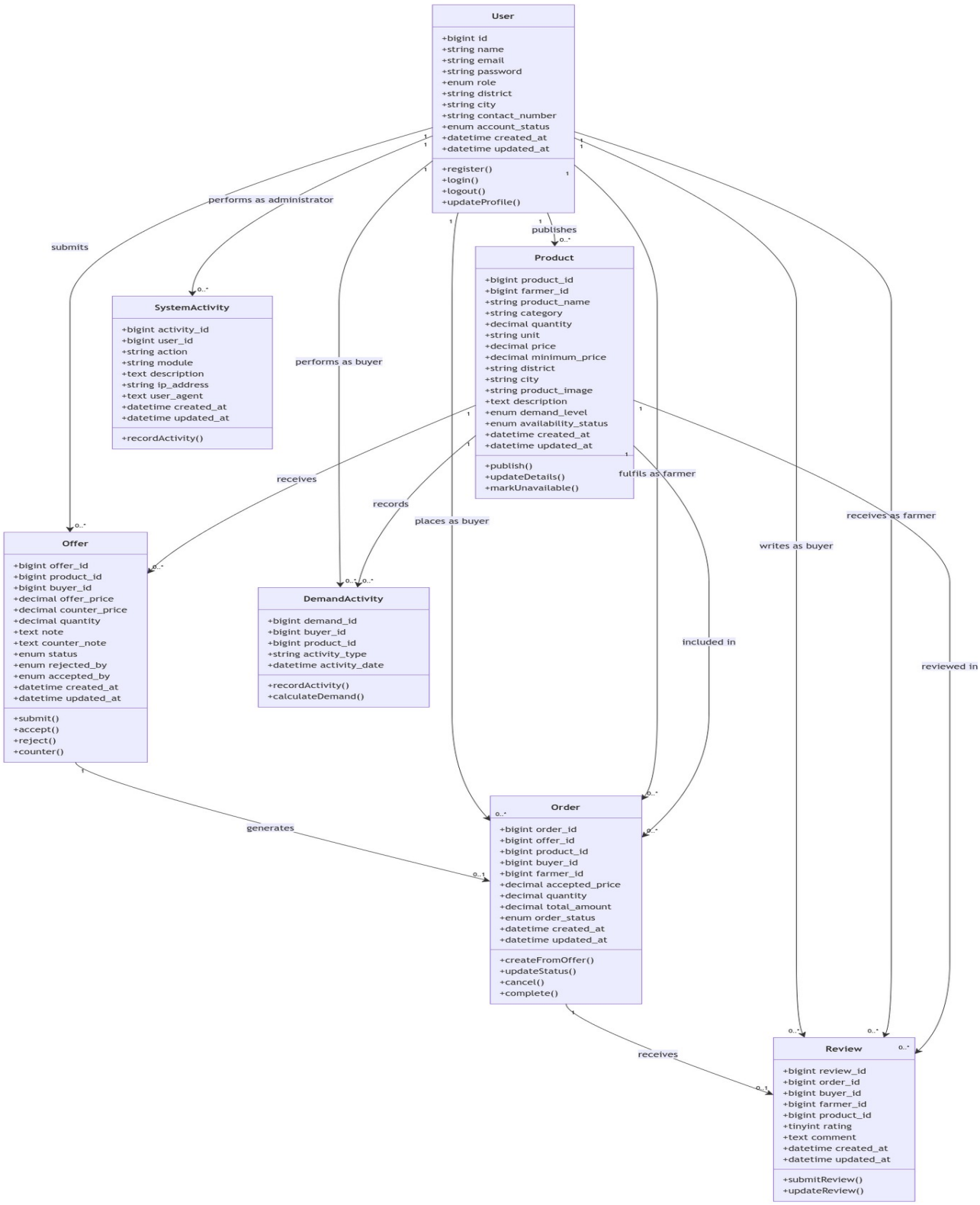



*Figure 3.5.1: Class Diagram*

# CHAPTER 4 – SYSTEM TESTING

## 4.1. Introduction

This chapter presents the testing activities performed to verify the functionality, security, usability, and overall performance of the AgroSmart Marketplace. Different testing techniques were applied to the main system modules, including user authentication, product management, offer negotiation, order processing, reviews, demand analysis, and administrative operations. The actual results were compared with the expected results to confirm that the implemented system operates correctly.

## 4.2. Functional Testing

Functional testing was conducted to verify that every system function operates according to the specified requirements.

| Test ID | Test Scenario | Test Data/Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| FT01 | Register a new farmer | Valid farmer details | Farmer account should be created | Account was created successfully | Pass |
| FT02 | Register a new buyer | Valid buyer details | Buyer account should be created | Account was created successfully | Pass |
| FT03 | Register with an existing email | Previously registered email | Registration should be rejected | Duplicate email error was displayed | Pass |
| FT04 | Login with valid credentials | Correct email and password | User should access the relevant dashboard | Relevant dashboard was displayed | Pass |
| FT05 | Login with invalid credentials | Incorrect password | Login should be rejected | Error message was displayed | Pass |
| FT06 | Access a page without login | Unauthenticated user | User should be redirected to login | User was redirected to login | Pass |

***Table 4.2.1: User Registration and Login Test Cases***

| Test ID | Test Scenario | Test Data/Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| FT07 | Publish a product | Valid product information | Product should be saved and displayed | Product was published successfully | Pass |
| FT08 | Publish with missing details | Required field left empty | Validation message should appear | Validation message was displayed | Pass |
| FT09 | Upload a product image | Valid image file | Image should be stored and displayed | Image was displayed successfully | Pass |
| FT10 | Update a product | Valid updated information | Product details should be updated | Details were updated successfully | Pass |
| FT11 | Mark product unavailable | Available product | Product status should change | Status changed successfully | Pass |
| FT12 | Browse available products | Logged-in buyer | Available products should be displayed | Products were displayed correctly | Pass |

***Table 4.2.2: Product Management Test Cases***

| Test ID | Test Scenario | Test Data/Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| FT13 | Submit an offer | Valid price and quantity | Offer should be sent to the farmer | Offer was submitted successfully | Pass |
| FT14 | Submit duplicate offer | Same buyer and product | Duplicate offer should be rejected | Error message was displayed | Pass |
| FT15 | Offer more than available quantity | Quantity exceeds stock | Offer should be rejected | Validation message was displayed | Pass |
| FT16 | Farmer accepts an offer | Pending offer | Offer status should become accepted | Status changed to accepted | Pass |
| FT17 | Farmer rejects an offer | Pending offer | Offer status should become rejected | Status changed to rejected | Pass |
| FT18 | Farmer sends a counteroffer | Valid counter price | Counteroffer should be shown to buyer | Counteroffer was displayed | Pass |
| FT19 | Buyer accepts counteroffer | Countered offer | Offer should be accepted | Counteroffer was accepted | Pass |
| FT20 | Buyer rejects counteroffer | Countered offer | Offer should be rejected | Counteroffer was rejected | Pass |

***Table 4.2.3: Offer Negotiation Test Cases***

| Test ID | Test Scenario | Test Data/Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| FT21 | Generate an order | Accepted offer | New order should be created | Order was created successfully | Pass |
| FT22 | Confirm an order | Pending order | Status should change to confirmed | Order was confirmed | Pass |
| FT23 | Mark order ready for pickup | Confirmed order | Status should change accordingly | Status was updated | Pass |
| FT24 | Complete an order | Ready-for-pickup order | Status should become completed | Order was completed | Pass |
| FT25 | Cancel an order | Permitted order status | Status should become cancelled | Order was cancelled | Pass |
| FT26 | Review a completed order | Rating and comment | Review should be saved | Review was saved successfully | Pass |
| FT27 | Review an incomplete order | Order not completed | Review submission should be blocked | Submission was blocked | Pass |

***Table 4.2.4: Order and Review Test Cases***

| Test ID | Test Scenario | Test Data/Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| FT28 | View registered users | Administrator login | User list should be displayed | Users were displayed successfully | Pass |
| FT29 | Deactivate a user account | Active user account | Account should be deactivated | Account was deactivated | Pass |
| FT30 | Login using deactivated account | Deactivated user | Login/access should be blocked | Access was blocked | Pass |
| FT31 | View system activities | Administrator login | Activity records should be displayed | Records were displayed correctly | Pass |

***Table 4.2.5: Administrator Test Cases***

## 4.3. Validation Testing

Validation testing was performed to ensure that invalid, incomplete, and incorrectly formatted data cannot be submitted to the system.

| Test ID | Input Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| VT01 | Empty required field | Validation message should appear | Validation message appeared | Pass |
| VT02 | Invalid email format | Email should be rejected | Invalid email was rejected | Pass |
| VT03 | Incorrect contact number | Contact number should be rejected | Validation error was displayed | Pass |
| VT04 | Negative product price | Product should not be saved | Invalid value was rejected | Pass |
| VT05 | Zero or negative quantity | Submission should be rejected | Validation error was displayed | Pass |
| VT06 | Unsupported image format | Image upload should be rejected | File was rejected | Pass |
| VT07 | Offer below permitted value | Offer should not be submitted | Validation message was displayed | Pass |

***Table 4.3.1: Validation Test Cases***

## 4.4. Security Testing

Security testing was conducted to ensure that user accounts and role-specific system functions are protected against unauthorized access.

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| ST01 | Access system without authentication | Redirect user to login | User was redirected | Pass |
| ST02 | Buyer accesses farmer page | Access should be denied | Access was denied | Pass |
| ST03 | Farmer accesses administrator page | Access should be denied | Access was denied | Pass |
| ST04 | Edit another farmer’s product | Operation should be blocked | Operation was blocked | Pass |
| ST05 | Respond to another buyer’s offer | Operation should be blocked | Operation was blocked | Pass |
| ST06 | Store a user password | Password should be encrypted | Encrypted password was stored | Pass |

***Table 4.4.1: Security Test Cases***

## 4.5. Usability and Responsive Testing

Usability and responsive testing was performed to evaluate the clarity, consistency, and adaptability of the user interfaces.

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| UT01 | Navigate using the sidebar | Pages should open correctly | Navigation worked correctly | Pass |
| UT02 | Submit a system form | Clear success/error feedback should appear | Appropriate message appeared | Pass |
| UT03 | View status badges | Status should be clearly identifiable | Status badges were displayed | Pass |
| UT04 | Use system on desktop | Layout should display correctly | Layout displayed correctly | Pass |
| UT05 | Use system on tablet | Layout should adapt to screen size | Layout adapted correctly | Pass |
| UT06 | Use system on mobile device | Content should remain accessible | Responsive layout was displayed | Pass |

***Table 4.5.1: Usability and Responsive Test Cases***

# CHAPTER 5 – CONCLUSION AND FUTURE WORK

## 5.1. Conclusion

The AgroSmart Marketplace was successfully developed as a web-based platform that connects farmers and buyers through a secure and convenient digital marketplace. The system allows farmers to publish and manage agricultural products, while buyers can browse available products, submit offers, negotiate prices, and place orders.

The main functions of the system, including role-based authentication, product management, offer negotiation, order processing, reviews and feedback, demand analysis, and administrative user management, were successfully implemented. The system was developed using Laravel, MySQL, Blade, Tailwind CSS, and Alpine.js by following the MVC architectural pattern.

Testing confirmed that the major system functions operate according to the specified requirements. Input validation, authentication, role-based authorization, and responsive interfaces improve the reliability, security, and usability of the system. Overall, the AgroSmart Marketplace achieves its main objective of providing an efficient platform for direct communication and trading between farmers and buyers.

## 5.2. Future Work

Although the current system provides the main marketplace functions, the following improvements can be introduced in the future:

- Integrate an online payment gateway for secure digital transactions.
- Add real-time chat between farmers and buyers.
- Provide email, SMS, and push notifications for offers and order updates.
- Integrate location services and maps to identify nearby products and farmers.
- Add delivery and shipment tracking facilities.
- Develop a mobile application for Android and iOS devices.
- Introduce multilingual support, including Sinhala and Tamil.
- Use artificial intelligence to predict product demand and recommend suitable prices.
- Provide advanced sales, demand, and user activity reports.
- Improve the review system with farmer and buyer reputation scores.
- Add stronger security features such as two-factor authentication.
- Integrate agricultural information such as weather forecasts and market price updates.

# References


- Laravel. (2026). *Laravel 13.x Documentation*. Available at: https://laravel.com/docs/13.x (Accessed: 22 July 2026).
- Laravel. (2026). *Laravel Jetstream Documentation*. Available at: https://jetstream.laravel.com (Accessed: 22 July 2026).
- PHP Documentation Group. (2026). *PHP Manual*. Available at: https://www.php.net/manual/en/ (Accessed: 22 July 2026).
- Oracle Corporation. (2026). *MySQL Reference Manual*. Available at: https://dev.mysql.com/doc/ (Accessed: 22 July 2026).
- Tailwind Labs. (2026). *Tailwind CSS Documentation*. Available at: https://tailwindcss.com/docs (Accessed: 22 July 2026).
- Alpine.js. (2026). *Alpine.js Documentation*. Available at: https://alpinejs.dev/start-here (Accessed: 22 July 2026).
- International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level Syllabus, Version 4.0.1*. Available at: ISTQB CTFL Syllabus (Accessed: 22 July 2026).
- diagrams.net. (2026). *draw.io – Diagramming Application*. Available at: https://app.diagrams.net (Accessed: 22 July 2026).
- Laravel. (2026). *Authentication – Laravel 13.x Documentation*. Available at: https://laravel.com/docs/13.x/authentication (Accessed: 22 July 2026).
- Laravel. (2026). *Authorization – Laravel 13.x Documentation*. Available at: https://laravel.com/docs/13.x/authorization (Accessed: 22 July 2026).
- Laravel. (2026). *Validation – Laravel 13.x Documentation*. Available at: https://laravel.com/docs/13.x/validation (Accessed: 22 July 2026).
- Laravel. (2026). *Laravel Fortify Documentation*. Available at: https://laravel.com/docs/13.x/fortify (Accessed: 22 July 2026).
- Laravel. (2026). *HTTP Testing – Laravel 13.x Documentation*. Available at: https://laravel.com/docs/13.x/http-tests (Accessed: 22 July 2026).
- OWASP Foundation. (2025). *OWASP Top 10:2025 – Web Application Security Risks*. Available at: https://owasp.org/Top10/2025/ (Accessed: 22 July 2026).